

PROMPT TO PROFIT™

WORKBOOK 05

Appointment Booking System

Build a Complete Appointment Scheduling
Application with AI

TECHNOLOGY STACK

HTML

CSS

Vanilla JavaScript

Supabase

DIFFICULTY

Beginner

ESTIMATED COMPLETION TIME

25-30 Hours

PRODUCED BY

Tochukwu Tech and AI Academy

www.tochukwunkwocha.com

PROMPT TO PROFIT™

Workbook 05 · Appointment Booking System

© 2026 Tochukwu Tech and AI Academy. All rights reserved.

Produced and published by Tochukwu Tech and AI Academy.

www.tochukwunkwocha.com

This workbook is designed for personal learning and guided software-building practice.

Technology names and product names remain the property of their respective owners.

Contents

Workbook 05 · Appointment Booking System

LEARNER SUPPORT TOOLKIT	5
VISUAL GLOSSARY	6
WORK SAFELY: MAKE A BACKUP	8
SOMETHING WENT WRONG – WHAT SHOULD I CHECK?	9
MY ERROR LOG	11
CHAPTER 1 · BUILDING THE PUBLIC WEBSITE	13
LESSON 1 · UNDERSTANDING THE APPOINTMENT BOOKING SYSTEM	14
LESSON 2 · BUILDING THE LANDING PAGE	18
LESSON 3 · STYLING THE LANDING PAGE	32
LESSON 4 · ADDING THE RESPONSIVE NAVIGATION	43
LESSON 5 · COMPLETING THE PUBLIC WEBSITE	52
CHAPTER SUMMARY	61
CHAPTER MILESTONE	62
CHAPTER 2 · CONNECTING TO SUPABASE	64
LESSON 1 · CREATING YOUR SUPABASE PROJECT	66
LESSON 2 · UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY	71
LESSON 3 · CREATING YOUR SUPABASE CONNECTION FILE	76
LESSON 4 · BUILDING A CONNECTION TEST PAGE	83
LESSON 5 · FINAL CONNECTION REVIEW	91
CHAPTER SUMMARY	96
CHAPTER MILESTONE	97
CHAPTER 3 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	98
LESSON 1 · UNDERSTANDING THE AUTHENTICATION FLOW	101
LESSON 2 · BUILDING THE COMPLETE AUTHENTICATION SYSTEM	105
LESSON 3 · AUDITING THE AUTHENTICATION SYSTEM	127

CHAPTER SUMMARY	132
CHAPTER MILESTONE	133
CHAPTER 4 · BUILDING THE APPOINTMENT DATABASE	134
LESSON 1 · UNDERSTANDING THE APPOINTMENT DATA MODEL	136
LESSON 2 · CREATING THE APPOINTMENTS TABLE	140
LESSON 3 · PROTECTING APPOINTMENT DATES, TIMES AND STATUSES	145
LESSON 4 · ENABLING ROW LEVEL SECURITY	149
LESSON 5 · CREATING THE SELECT POLICY	152
LESSON 6 · CREATING THE INSERT POLICY	156
LESSON 7 · CREATING THE UPDATE POLICY	160
LESSON 8 · CREATING SECURE APPOINTMENT CANCELLATION	165
LESSON 9 · TESTING THE APPOINTMENT DATABASE	169
CHAPTER SUMMARY	173
CHAPTER MILESTONE	174
CHAPTER 5 · BUILDING THE APPOINTMENT DASHBOARD	175
LESSON 1 · DESIGNING THE APPOINTMENT DASHBOARD	176
LESSON 2 · BUILDING THE APPOINTMENT BOOKING FORM	185
LESSON 3 · SAVING APPOINTMENTS SECURELY	191
LESSON 4 · BUILDING APPOINTMENT DASHBOARD STATISTICS	197
CHAPTER SUMMARY	202
CHAPTER MILESTONE	203
CHAPTER 6 · VIEWING APPOINTMENTS	204
LESSON 1 · BUILDING THE APPOINTMENT SCHEDULE	205
LESSON 2 · BUILDING THE APPOINTMENT DETAILS PAGE	210
LESSON 3 · TESTING THE APPOINTMENT SCHEDULE	216
CHAPTER SUMMARY	220
CHAPTER MILESTONE	221
CHAPTER 7 · SEARCHING, FILTERING AND SORTING APPOINTMENTS	222
LESSON 1 · BUILDING APPOINTMENT SEARCH, FILTERING AND SORTING	224
LESSON 2 · TESTING APPOINTMENT DISCOVERY	229
CHAPTER SUMMARY	232

CHAPTER MILESTONE	233
CHAPTER 8 · EDITING APPOINTMENTS	234
LESSON 1 · BUILDING APPOINTMENT EDITING	236
LESSON 2 · TESTING APPOINTMENT EDITING	242
CHAPTER SUMMARY	245
CHAPTER MILESTONE	246
CHAPTER 9 · CANCELLING APPOINTMENTS	247
LESSON 1 · BUILDING SECURE APPOINTMENT CANCELLATION	248
LESSON 2 · TESTING APPOINTMENT CANCELLATION	253
CHAPTER SUMMARY	256
CHAPTER MILESTONE	257
CHAPTER 10 · FINAL APPLICATION REVIEW	258
LESSON 1 · COMPLETE SYSTEM TESTING	260
REFLECTION QUESTIONS	267
EXTENSION CHALLENGES	268
NEXT WORKBOOK	269

ABOUT THIS WORKBOOK

This workbook is a complete, self-contained project.

You do not need to own any of the other Prompt to Profit™ Software Workbooks to complete it successfully.

Every workbook in this series teaches you how to build one complete software application from start to finish. It contains all the explanations, prompts, instructions and practical exercises you need to complete that project independently.

Although some software development concepts naturally appear across multiple workbooks, every workbook starts from the beginning of its own project and assumes no prior knowledge of the other workbooks.

This means you can study the workbooks in any order that suits your goals.

If your immediate need is to build an Appointment Booking System, you can begin with this workbook.

If you later decide to build additional software, each new workbook will teach you a different business application while reinforcing the software development skills you have already learned.

Together, the workbooks form a complete software development series. Individually, each workbook is a complete learning experience.

WELCOME

Welcome to Prompt to Profit™ Workbook 05.

In this workbook, you will build a complete Appointment Booking System from the ground up using HTML, CSS, Vanilla JavaScript, Supabase and Artificial Intelligence.

Throughout this project, you will learn how to build secure, database-driven business software by following the same step-by-step development process used in real-world software projects.

You will create a professional appointment management application that allows businesses to register users, manage appointment information securely, search and organise appointments, update appointment details, and protect every user's data using Row Level Security.

Every feature is built one step at a time. You won't simply copy prompts into an AI tool. You'll learn why each feature is being built, how the different parts of the application work together, and how to communicate effectively with AI to build reliable software.

By the end of this workbook, you will have a fully functional Appointment Booking System that you can proudly include in your software development portfolio.

You will build the project using:

- HTML

- CSS
- Vanilla JavaScript
- Supabase
- ChatGPT
- Notepad
- A web browser

You do not need to install additional software.

You will continue working with complete files and clear Build Prompts.

Every new feature will preserve everything already built.

Complete the lessons in order.

Test each feature before moving forward.

Do not continue when an important feature is not working.

WHAT YOU WILL BUILD

By the end of this workbook, your Appointment Booking System will include:

PUBLIC WEBSITE

- Responsive navigation
- Hero, problem, features and How It Works sections
- Sample schedule preview
- Login and Register links

SUPABASE

- Supabase project and shared connection file
- Secure appointments table
- Date, time, status and overlap protection

AUTHENTICATION

- Registration and email verification
- Login, protected pages and logout

APPOINTMENT MANAGEMENT

- Book appointments
- View a date-grouped schedule
- Open complete appointment details
- Search, filter and sort appointments
- Edit appointments
- Cancel appointments without deleting their history

BUSINESS DASHBOARD

- Total, today's, upcoming and cancelled appointments

SECURITY AND COMPLETION

- Row Level Security and authenticated ownership checks
- Two-account privacy testing
- Netlify deployment
- Portfolio description, reflection and extension challenges

WORKBOOK STRUCTURE

This workbook is organised into ten chapters.

Chapter 1

Building the Public Website

Chapter 2

Connecting to Supabase

Chapter 3

Building the Complete Authentication System

Chapter 4

Building the Appointment Database

Chapter 5

Building the Appointment Dashboard

Chapter 6

Viewing Appointments

Chapter 7

Building the Appointment Details Page

Chapter 8

Editing and Deleting Appointment Records

Chapter 9

Making Appointment Management More Powerful

Chapter 10

Final Testing and Project Completion

Complete each chapter before moving to the next.

Every chapter depends on the work completed earlier.

LEARNER SUPPORT TOOLKIT

Keep this part of the workbook close while you build.

It explains common words, shows you how to protect your work, gives you a simple way to investigate problems, and provides a place to record errors and solutions.

You are not expected to remember everything immediately.

Return to these pages whenever you need them.

VISUAL GLOSSARY

HTML	The file that gives a web page its content and structure.
CSS	The file that controls colours, spacing, layout and appearance.
VANILLA JAVASCRIPT	JavaScript used without a framework. It controls what the application does.
BROWSER	The program used to open and test the application, such as Chrome or Edge.
FILE	One saved part of the project, such as index.html or dashboard.js.
FOLDER	The place where all files for one project are kept together.
FUNCTION	A named set of instructions that performs one task.
EVENT	An action the browser can notice, such as a click, form submission or key press.
VARIABLE	A named place used to hold information while the application is running.
URL	The address of a page, website or online service.
SUPABASE	The online service used for the database, authentication and security.
DATABASE	An organised place where the application stores information.
TABLE	A named part of the database that stores one type of information.
RECORD	One complete item saved inside a database table.
AUTHENTICATION	The process of creating accounts, signing in and identifying the current user.
SESSION	The saved sign-in state that allows the application to remember the current user.
ROW LEVEL SECURITY	Supabase rules that control which database records each user can access.

PUBLISHABLE KEY	The Supabase project key that browser-based applications are allowed to use.
DEPLOY	To place the complete project online so it has a live HTTPS website address.
DEBUGGING	The careful process of finding, understanding and correcting a problem.

WORK SAFELY: MAKE A BACKUP

A backup is a separate copy of your complete project folder.

Create a backup before replacing files that already contain working code.

1.

Close any project files that are open in Notepad.

2.

Find your complete project folder.

3.

Copy the folder.

4.

Paste the copy outside the working project folder.

Do not place the backup inside the folder you deploy.

5.

Rename the copy clearly.

For example:

Project Backup - Before Chapter 3 Lesson 2

6.

Open the backup folder.

Confirm that the expected files are inside it.

If a new change causes a serious problem, you can return to this working copy.

SOMETHING WENT WRONG — WHAT SHOULD I CHECK?

Problems are a normal part of building software.

Do not replace random pieces of code and do not continue to the next lesson.

Work through this checklist in order.

- ☐ Confirm that you opened the correct project folder.
- ☐ Confirm that every updated file was saved in Notepad.
- ☐ Check every filename carefully.
- ☐ Make sure an HTML, CSS or JavaScript file does not end with .txt.
- ☐ Confirm that you pasted the complete file returned by AI.
- ☐ Confirm that you did not paste an explanation, a code-block label or only part of a file.
- ☐ Check that stylesheet and JavaScript filenames match the names used inside the HTML.
- ☐ On Supabase pages, check that the scripts load in the order taught in the lesson.
- ☐ Read the exact message shown on the page.
- ☐ Open the browser Console and look for the first red error.

To open the Console:

Right-click the page.

Select:

Inspect

Then select:

Console

- ☐ Copy the complete first red error into your error log.
- ☐ Think about the last file you changed before the problem appeared.
- ☐ Compare that file with the lesson requirements.
- ☐ If necessary, restore the most recent working backup.

When asking AI for help, provide:

-
- What you were trying to do
 - What you expected to happen
 - What actually happened
 - The complete error message
 - The names of the files involved

Ask AI to return complete corrected files.

Do not ask for snippets.

MY ERROR LOG

Use this page whenever something does not work.

Writing down the problem helps you investigate it carefully and remember the solution.

ERROR 1

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

ERROR 2

Date: _____

Chapter and lesson: _____

What I was trying to do:

What I expected to happen:

What actually happened:

First error message:

File or files involved:

What I checked or changed:

What fixed the problem:

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

CHAPTER INTRODUCTION

Every useful business application begins with a clear purpose.

An Appointment Booking System helps a business organise time, reduce missed bookings and understand what is happening each day.

Before building the private booking area, you will create a simple public website that explains the problem and introduces the solution.

This chapter begins gently. You will create the project folder, build the landing page, style it and add responsive navigation.

You will continue using only HTML, CSS, Vanilla JavaScript, Notepad and a web browser.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- A clear public landing page
- A hero section for the Appointment Booking System
- A business problem section
- A features section
- A simple How It Works section
- A sample schedule preview
- Login and Register links
- Responsive navigation

By the end of the chapter, your project will have a complete public website ready to connect to Supabase.

LESSON 1

UNDERSTANDING THE APPOINTMENT BOOKING SYSTEM

Estimated Time

10 minutes

WHAT YOU ARE BUILDING

You are building software that helps a business manage appointment information in one organised system.

A user will be able to create an account and store appointment details such as:

- Name
- Phone number
- Email address
- Service
- Appointment date and time
- Appointment Date
- Start and end time
- Notes

The software will also allow the user to search, filter, sort, update, and cancel appointments.

Every user's appointment information will remain private.

WHY THIS MATTERS

Many small businesses do not have a reliable appointment system.

Appointment information may be stored inside phone contacts, notebooks, messaging applications, spreadsheets, receipts, or the memory of one employee.

This creates several problems.

The business may struggle to find an appointment's number.

Two employees may create the same appointment.

Important notes may be forgotten.

Appointment information may be lost when a phone is replaced.

The business may also find it difficult to understand how many appointments it has or how quickly the appointment list is growing.

A Appointment Booking System solves these problems by keeping appointment information in one structured database.

BEFORE YOU CONTINUE

Create a new folder on your computer.

Name it exactly:

Appointment Booking System

This folder will contain every file used throughout Workbook 05.

Do not copy or save files from any other workbook inside this folder.

Each software project should have its own folder.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Create the project folder and make sure you understand what the completed application will do.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson prepares you for the project.

SAVE YOUR FILES

No files are created during this lesson.

Confirm that the folder is named:

Appointment Booking System

TEST YOUR WORK

Open the new folder.

It should be empty.

Do not copy files from any other workbook into this folder.

You will build this project from the beginning.

CHECKPOINT

Before moving on, confirm that:

- ☒ You created the Appointment Booking System folder.
- ☒ The folder is empty.
- ☒ You understand the main purpose of the software.
- ☒ You understand that every user will have private appointments.

COMMON BEGINNER MISTAKES

One common mistake is placing files from different projects inside the same folder.

This can cause the browser to load the wrong stylesheet or JavaScript file.

Keep this project completely separate from any other workbook.

Another mistake is renaming the folder repeatedly during the project.

Choose the correct name now and keep it unchanged.

BEHIND THE SCENES

The application will eventually contain several connected systems.

The public website will explain the software.

Authentication will identify the user.

The appointments table will store records.

Row Level Security will protect those records.

JavaScript will connect the forms and dashboard to Supabase.

Every part will be added in the correct order.

THINK LIKE A SOFTWARE DESIGNER

Before building software, understand the real problem it is expected to solve.

This project is not simply a form that saves names and client phone numbers.

It is a business system designed to make appointment information easier to store, find, update, and use.

That difference should guide every feature you build.

WHAT YOU LEARNED

In this lesson you learned:

- what an Appointment Booking System does
- why businesses need organised appointments
- what information the software will store
- why every software project should have its own folder
- why the application must protect each user's data

In the next lesson, you will build the first version of the public landing page.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 2

BUILDING THE LANDING PAGE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the first version of the public website for your Appointment Booking System.

The landing page will explain what the software does, the business problems it solves, and the main features users will receive.

At this stage, the page will contain the complete HTML structure.

It will not look polished yet because the stylesheet has not been created.

The responsive navigation will also not work until the JavaScript file is added in a later lesson.

WHY THIS MATTERS

A business software application needs a clear public website.

Before someone creates an account, they should be able to understand:

- What the software does
- Who it is designed for
- What problems it solves
- What features are available
- What action they should take next

A well-structured landing page helps visitors understand the value of the application before they register.

BEFORE YOU CONTINUE

Confirm that your project folder is named:

Appointment Booking System

Open the folder.

It should still be empty.

You are about to create the first file:

index.html

When saving the file in Notepad:

- Click File.
- Click Save As.
- Open the Appointment Booking System folder.
- Change Save as type to All Files.
- Enter the filename exactly as:

index.html

Make sure the file is not saved as:

index.html.txt

BUILD PROMPT

PROMPT STARTS HERE

Create one complete HTML5 file named:

index.html

The file is the public landing page for an Appointment Booking System designed for small businesses.

Return the complete index.html file only.

Do not return explanations.

Do not return snippets.

Do not return CSS.

Do not return JavaScript.

Do not omit any section.

FILE CONNECTIONS

Inside the head section, include this stylesheet link exactly:

```
<link rel="stylesheet" href="styles.css">
```

Immediately before the closing body tag, include this JavaScript link exactly:

```
<script src="script.js"></script>
```

The files styles.css and script.js will be created in later lessons.

Do not include CSS inside index.html.

Do not include JavaScript inside index.html.

GENERAL HTML REQUIREMENTS

- Use proper HTML5 structure.
- Include the viewport meta tag.
- Include an appropriate page title.
- Use semantic HTML elements.
- Use clear and meaningful class names.
- Use section IDs that match the navigation links.
- Keep the content professional and easy to understand.
- Do not use external images.
- Do not use external icons.
- Do not use any framework.
- Do not use placeholder text such as Lorem Ipsum.

PAGE STRUCTURE

Build the sections below in this exact order.

1. HEADER AND NAVIGATION

Create a professional header.

Include a clickable logo that displays:

Appointment Records

The logo must link to:

index.html

Include these navigation links:

Home

Features

How It Works

Login

Register

The links should point to:

Home:

#home

Features:

#features

How It Works:

#how-it-works

Login:

login.html

Register:

register.html

Style-related classes should already be added so CSS can later make Login a secondary action and Register the main action.

Add a mobile hamburger button.

The hamburger button must:

- Use a button element.
- Include exactly three empty span elements.
- Include an aria-label that explains its purpose.
- Include:

aria-expanded="false"

- Include:

aria-controls

pointing to the navigation menu.

Do not build the hamburger lines using CSS pseudo-elements.

The three span elements must exist inside the HTML.

2. HERO SECTION

Give the section the ID:

home

Include:

- A clear headline explaining that the software helps businesses organise appointment information.
- A short supporting paragraph.
- A primary button labelled:

Get Started

The button must link to:

register.html

- A secondary button labelled:

See How It Works

The button must link to:

#how-it-works

Also include an appointment management preview built entirely with HTML.

The preview should contain simple cards or panels representing:

- Total Appointments
- Upcoming Appointments
- Cancelled Appointments
- A short appointment list

Do not use an image.

3. BUSINESS PROBLEM SECTION

Create a section explaining the problems businesses face when appointment information is scattered across:

- Notebooks
- Phone contacts
- Spreadsheets
- Messaging applications

- Employees' personal devices

Explain that scattered records can lead to:

- Lost appointment details
- Duplicate records
- Missed follow-ups
- Slow service
- Difficulty understanding appointment activity

Keep the wording clear and concise.

4. FEATURES SECTION

Give the section the ID:

features

Include six feature cards.

The cards should explain:

1. Secure Appointment Records
2. Overlap Protection
3. Appointment Details Cards
4. Click-to-Call
5. Click-to-Email
6. Search, Filters and Sorting

Each card should include:

- A short title
- A brief explanation

Do not use external icons.

You may use simple text labels or small decorative HTML elements.

5. HOW IT WORKS SECTION

Give the section the ID:

how-it-works

Explain the process in four clear steps:

1. Create an Account

2. Book Appointment Details

3. Manage Appointment Records

4. Find and Contact Appointments Quickly

Each step should have:

- A number
- A heading
- A short explanation

6. DASHBOARD PREVIEW SECTION

Create a larger dashboard preview using HTML only.

Include:

- Four summary cards
- An appointment search area
- Filter controls
- Three sample appointment cards

Each sample appointment card should display example information such as:

- Client initials
- Client name
- Service
- Service
- Phone
- Email
- View Appointment button

The sample data must be clearly fictional.

Do not use real personal information.

The buttons are only part of the visual preview and do not need to work.

7. FINAL CALL-TO-ACTION SECTION

Encourage the visitor to organise appointments in one secure system.

Include a button labelled:

Create Your Account

The button must link to:

register.html

8. FOOTER

Include:

- Logo or software name
- A short description
- Quick links
- Login link pointing to login.html
- Register link pointing to register.html
- Copyright text

Use a JavaScript-friendly span with an ID for the copyright year if appropriate, but do not include JavaScript inside the page.

ACCESSIBILITY REQUIREMENTS

- Use heading levels in a logical order.
- Add descriptive text to links and buttons.
- Use aria-labels where necessary.
- Make the hamburger button accessible.
- Use meaningful link text.
- Avoid empty links.

IMPORTANT

The Login links must point to:

login.html

The Register and Get Started links must point to:

register.html

The stylesheet link must be:

```
<link rel="stylesheet" href="styles.css">
```

The JavaScript link must be:

```
<script src="script.js"></script>
```

Return one complete index.html file from beginning to end.

Do not include unfinished comments.

Do not return any other file.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

index.html

The file should contain:

- ☒ Correct HTML5 structure
- ☒ Stylesheet link to styles.css
- ☒ JavaScript link to script.js
- ☒ Header and navigation
- ☒ Three-span hamburger button
- ☒ Hero section
- ☒ Business problem section
- ☒ Six feature cards
- ☒ Four-step How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer
- ☒ Login links pointing to login.html
- ☒ Register links pointing to register.html

The response should begin with:

<!DOCTYPE html>

It should not begin with an explanation.

SAVE YOUR FILES

Copy the complete HTML code returned by ChatGPT.

Open Notepad.

Paste the complete code into the empty document.

Click File.

Click Save As.

Open your Appointment Booking System folder.

Choose:

Save as type: All Files

Enter the filename exactly as:

index.html

Click Save.

Close Notepad.

Open the project folder and confirm that the file appears as:

index.html

Make sure it is not named:

index.html.txt

TEST YOUR WORK

Open your Appointment Booking System folder.

Double-click:

index.html

The page should open in your browser.

It will look plain because styles.css does not exist yet.

That is expected.

Scroll through the complete page.

Confirm that you can see:

- ☒ Header
- ☒ Navigation
- ☒ Hero section
- ☒ Business problem section
- ☒ Features section
- ☒ How It Works section
- ☒ Dashboard preview
- ☒ Final call-to-action
- ☒ Footer

Click:

Features

The browser should move to the Features section.

Click:

How It Works

The browser should move to the How It Works section.

Click:

Login

The browser may show that login.html does not exist.

This is expected.

Click:

Register

The browser may show that register.html does not exist.

This is also expected.

Those pages will be created later as part of the complete authentication system.

CHECKPOINT

Before moving on, confirm that:

- ✓ index.html exists.
- ✓ The browser opens the file as a webpage.
- ✓ The page does not display HTML code as plain text.
- ✓ Every required section is present.
- ✓ The hamburger button contains three span elements.
- ✓ Login points to login.html.
- ✓ Register points to register.html.
- ✓ styles.css is linked.
- ✓ script.js is linked.
- ✓ The filename is not index.html.txt.

COMMON BEGINNER MISTAKES

A common mistake is saving the file using the wrong extension.

If the file is saved as:

index.html.txt

the browser may not treat it as a webpage.

Open the project folder and check the complete filename carefully.

Another mistake is removing the links to styles.css and script.js because those files do not exist yet.

Do not remove them.

They are being linked now so the files will connect correctly when they are created.

Some students also change:

login.html

to:

login

or change:

register.html

to:

signup.html

Do not do this.

The filenames must remain consistent because later pages and redirects will depend on them.

BEHIND THE SCENES

The landing page already refers to files that will be created in later lessons.

This is intentional.

The browser reads:

```
<link rel="stylesheet" href="styles.css">
```

and looks for a file named styles.css inside the same folder.

It also reads:

```
<script src="script.js"></script>
```

and looks for script.js inside the same folder.

The page remains usable even though those files are not present yet, but it will not have its final appearance or interactive navigation until they are created.

THINK LIKE A SOFTWARE DESIGNER

A public website should help visitors understand the software before asking them to create an account.

The sections should answer the visitor's questions in a sensible order.

First, explain the software.

Next, explain the business problem.

Then show the features and how the system works.

Finally, ask the visitor to register.

This creates a clear journey instead of presenting disconnected information.

CODE-READING QUESTION

Open index.html. Find the part of the page added for building the landing page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create the main HTML file for a business application
- structure a professional landing page
- connect future CSS and JavaScript files correctly
- prepare navigation for future authentication pages
- build a hamburger button with three span elements
- save an HTML file correctly using Notepad

In the next lesson, you will create styles.css and transform the plain landing page into a polished, responsive business website.

CHAPTER 1**BUILDING THE PUBLIC WEBSITE****LESSON 3**

STYLING THE LANDING PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the stylesheet for your Appointment Booking System.

The HTML structure is already complete.

You will now transform it into a professional business website suitable for a real software product.

The responsive navigation will be completed in the next lesson when you create the JavaScript file.

WHY THIS MATTERS

Without CSS, a webpage contains only structure.

CSS controls:

- Colours
- Typography
- Layout
- Spacing
- Buttons
- Cards
- Responsive behaviour

A professional design helps visitors understand the software more quickly and creates confidence in the product.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ✓ index.html opens correctly.
- ✓ Every landing page section appears.
- ✓ The browser does not display HTML code as plain text.

Open index.html in Notepad.

Copy the complete code.

You will paste it into the Build Prompt so ChatGPT can style the exact HTML structure you have already created.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete HTML file named:

index.html

for my Appointment Booking System.

Create one complete CSS file named:

styles.css

Return only the complete styles.css file.

Do not return HTML.

Do not return JavaScript.

Do not return explanations.

Do not return snippets.

Everything already built must continue working.

The stylesheet must match the exact HTML structure that I will paste below.

The HTML already contains this stylesheet link:

```
<link rel="stylesheet" href="styles.css">
```

GENERAL DESIGN

Create a clean, modern business software design.

Use a professional colour palette suitable for business applications.

Create CSS variables inside:

:root

Include variables for:

- Primary colour
- Secondary colour
- Accent colour
- Background colour
- Card colour
- Text colour
- Border colour
- Shadow
- Border radius
- Content width

Use:

box-sizing: border-box;

for every element.

Remove unnecessary browser margins and padding.

Use a professional system font stack.

Do not import fonts.

GENERAL LAYOUT

Create a centred content container.

Give each major section comfortable spacing.

Prevent content from touching the edges of the screen.

Create balanced spacing between headings, paragraphs, cards and buttons.

HEADER

Create a professional header.

Display:

Appointment Records

clearly as the brand.

Arrange navigation horizontally on larger screens.

Style:

Login

as a secondary button.

Style:

Register

as the primary action.

Include hover and keyboard focus styles.

HAMBURGER BUTTON

Style the existing hamburger button.

The button already contains exactly three span elements.

Do not create additional lines using CSS pseudo-elements.

Hide the hamburger button on larger screens.

Display it on smaller screens.

Prepare the navigation menu so it becomes visible whenever JavaScript adds an:

active

class.

Do not write JavaScript.

HERO SECTION

Create an attractive hero layout.

Display the content and dashboard preview side by side on wider screens.

Stack them on smaller screens.

Make:

Get Started

the strongest visual button.

Style:

See How It Works

as a secondary button.

BUSINESS PROBLEM SECTION

Give this section a subtle visual difference from the hero section.

Use generous spacing.

Make the problem easy to read.

FEATURES SECTION

Display the six feature cards in a responsive grid.

Each card should include:

- Rounded corners
- Professional shadow
- Comfortable spacing
- Subtle hover effect

HOW IT WORKS SECTION

Display the four steps using attractive cards.

Clearly distinguish the step number from the description.

DASHBOARD PREVIEW

Style the HTML dashboard preview so it resembles a real business dashboard.

Style:

- Summary cards
- Search area
- Appointment cards
- View Appointment buttons

Do not depend on images.

FINAL CALL-TO-ACTION

Create a strong closing section.

Highlight the:

Create Your Account

button.

FOOTER

Create a professional footer.

Arrange footer content neatly.

Style footer navigation links.

RESPONSIVE DESIGN

Create responsive layouts for:

Desktop

Tablet

Mobile

On smaller screens:

Hide the desktop navigation.

Display the hamburger button.

Prepare the navigation to become visible when JavaScript adds the:

active

class.

Stack multi-column layouts vertically.

Prevent horizontal scrolling.

Keep buttons large enough to tap comfortably.

ACCESSIBILITY

Create visible keyboard focus styles.

Maintain good colour contrast.

Respect reduced-motion preferences where appropriate.

TECHNICAL REQUIREMENTS

Match the exact class names used in the supplied HTML.

Do not invent a different HTML structure.

Do not use CSS frameworks.

Do not use external fonts.

Return one complete:

styles.css

file.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

styles.css

The file should:

- ☒ Match the HTML.
- ☒ Style every section.
- ☒ Support desktop, tablet and mobile layouts.
- ☒ Prepare the navigation for JavaScript.
- ☒ Include no HTML.
- ☒ Include no JavaScript.

SAVE YOUR FILES

Copy the complete CSS returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Appointment Booking System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

styles.css

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

TEST YOUR WORK

Open:

index.html

If it is already open in your browser, refresh the page.

The landing page should now have a professional appearance.

Confirm that:

- ☒ Header is styled.
- ☒ Navigation is styled.
- ☒ Hero section looks professional.
- ☒ Dashboard preview resembles business software.
- ☒ Feature cards display correctly.
- ☒ Buttons look professional.
- ☒ Footer is styled.

Now reduce the browser width.

Confirm that:

- ☒ Layout adjusts correctly.

☒ Hamburger button appears.

☒ Desktop navigation disappears.

The hamburger button does not need to work yet.

That functionality will be added in the next lesson.

CHECKPOINT

Before moving on, confirm that:

☒ styles.css exists.

☒ The landing page is styled.

☒ Responsive layouts work.

☒ Hamburger button appears on smaller screens.

☒ Buttons display correctly.

☒ Dashboard preview looks professional.

☒ Footer is styled.

COMMON BEGINNER MISTAKES

A common mistake is saving the file as:

styles.css.txt

Always choose:

Save as type:

All Files

Another common mistake is changing class names inside the CSS.

The stylesheet must match the HTML structure exactly.

If ChatGPT returns styles that do not match your HTML, provide your complete index.html again and ask for a complete updated styles.css file.

Do not ask for snippets.

BEHIND THE SCENES

When your browser opens index.html, it immediately reads:

```
<link rel="stylesheet" href="styles.css">
```

It then loads every CSS rule inside styles.css and applies those rules to matching HTML elements.

This is why HTML and CSS must always stay in sync.

If an HTML class changes but the CSS is not updated, the styling may stop working.

THINK LIKE A SOFTWARE DESIGNER

Business software should communicate trust.

Professional colours, balanced spacing, consistent buttons, and organised layouts help users feel confident before they even create an account.

Good design is not decoration.

Good design makes software easier to understand and easier to use.

CODE-READING QUESTION

Open styles.css. Find one style rule added for styling the landing page. Which selector does the rule use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a professional stylesheet
- style a business landing page
- prepare responsive layouts

- style dashboard preview cards
- prepare the navigation for JavaScript
- keep HTML and CSS working together

In the next lesson, you will create `script.js` and make the responsive navigation fully functional.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 4

ADDING THE RESPONSIVE NAVIGATION

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the JavaScript that makes the responsive navigation work.

Your landing page already contains:

- HTML structure
- Professional styling
- A three-line hamburger button

The only missing piece is the behaviour.

When the browser window becomes narrow, visitors should be able to open and close the navigation menu using the hamburger button.

WHY THIS MATTERS

Many people will visit your website using a mobile phone or tablet.

A navigation menu that works well on a large computer screen may become difficult to use on a smaller device.

Responsive navigation solves this problem by allowing users to open and close the menu whenever they need it.

This creates a cleaner layout and improves the overall user experience.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that your project folder contains:

index.html

styles.css

Open index.html in your browser.

Reduce the browser width.

You should see:

☒ The hamburger button.

☒ The normal navigation hidden.

The hamburger button will not do anything yet.

That is expected.

Open index.html in Notepad.

Copy the complete code.

Also open styles.css in Notepad and copy the complete code.

You will paste both files into the Build Prompt so ChatGPT can use the exact IDs and class names already in your project.

BUILD PROMPT

PROMPT STARTS HERE

I already have these files in my project:

index.html

styles.css

Everything already built must continue working.

Create one complete JavaScript file named:

script.js

Return only the complete script.js file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

- Use plain Vanilla JavaScript.
- Do not use any framework.
- Do not use external libraries.
- Do not use Supabase.
- Do not add authentication code.
- Do not book appointment management code.

RESPONSIVE NAVIGATION

- Use the exact HTML structure that I provide.
- Find:
 - The hamburger button.
 - The navigation menu.
- Use the exact IDs or class names already present.
- When the hamburger button is clicked:
 - Open the navigation menu.
 - Add the existing active class expected by styles.css.
- When the hamburger button is clicked again:
 - Close the navigation menu.
 - Remove the active class.

ACCESSIBILITY

- Update:
 - aria-expanded
 - correctly.
- When the menu opens:
 - aria-expanded="true"
- When the menu closes:
 - aria-expanded="false"
- Do not remove any accessibility attributes already present.

CLOSING THE MENU

When the visitor clicks any navigation link inside the mobile menu:

- Close the menu.

When the visitor presses the Escape key:

- Close the menu.

When the visitor clicks anywhere outside the navigation:

- Close the menu.

When the browser window becomes wider than the mobile layout:

- Close the mobile menu automatically.

SMOOTH NAVIGATION

Allow links pointing to sections inside:

`index.html`

to move smoothly.

Do not interfere with links pointing to:

`login.html`

`register.html`

Those links must continue opening normally.

CODE QUALITY

Wait until the page has loaded before selecting HTML elements.

Check that required elements exist before attaching event listeners.

Use clear variable names.

Avoid duplicate event listeners.

Do not include unfinished comments.

Return one complete `script.js` file.

IMPORTANT

Everything already built must continue working.

Use the exact IDs and class names already present inside my HTML.

Do not invent different selectors.

The active class must match the class already prepared inside styles.css.

Here is my complete index.html file:

[PASTE YOUR COMPLETE INDEX.HTML HERE]

Here is my complete styles.css file:

[PASTE YOUR COMPLETE STYLES.CSS HERE]

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

script.js

The file should:

- ☒ Open the mobile navigation.
- ☒ Close the mobile navigation.
- ☒ Update aria-expanded.
- ☒ Close after selecting a navigation link.
- ☒ Close after pressing Escape.
- ☒ Close after clicking outside the navigation.
- ☒ Close automatically when returning to desktop width.
- ☒ Leave Login and Register links working normally.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Appointment Booking System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

script.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

Make sure the file is not named:

script.js.txt

TEST YOUR WORK

Open:

index.html

If it is already open, refresh the browser.

Reduce the browser width until the hamburger button appears.

Click the hamburger button.

The navigation menu should open.

Click the hamburger button again.

The navigation menu should close.

Open the menu.

Click:

Features

The menu should close automatically.

The browser should move to the Features section.

Open the menu again.

Press the Escape key.

The menu should close.

Open the menu again.

Click outside the navigation.

The menu should close.

Finally, widen the browser window.

The mobile menu should close automatically and the normal desktop navigation should appear again.

Click:

Login

The browser may display a page-not-found message.

This is expected because login.html has not yet been created.

Click:

Register

The browser may also display a page-not-found message.

This is expected because register.html will be created later in Workbook 05.

CHECKPOINT

Before moving on, confirm that:

- ☒ script.js exists.
- ☒ The hamburger button opens the menu.
- ☒ The hamburger button closes the menu.
- ☒ Navigation links close the menu.
- ☒ Escape closes the menu.

- ✓ Clicking outside closes the menu.
- ✓ Returning to desktop width closes the mobile menu.
- ✓ Login still points to login.html.
- ✓ Register still points to register.html.

COMMON BEGINNER MISTAKES

A common mistake is changing the class name used by JavaScript without updating the CSS.

The JavaScript and CSS must both use the same active class.

Another common mistake is searching for HTML elements using IDs or class names that do not exist.

Always use the exact HTML already present in your project.

If the menu does not open, provide ChatGPT with both your complete index.html and styles.css files and ask it to regenerate the complete script.js file.

Do not ask for only a small correction.

BEHIND THE SCENES

JavaScript does not create a second navigation menu.

Instead, it changes the state of the menu that already exists inside index.html.

When the active class is added, CSS displays the menu.

When the active class is removed, CSS hides it again.

The browser simply switches between those two states.

THINK LIKE A SOFTWARE DESIGNER

Responsive navigation should feel natural.

Users should never wonder how to close the menu or whether the application has stopped responding.

Good software provides predictable behaviour in every situation.

Small details like automatically closing the menu after a navigation link is selected make an application feel much more polished.

CODE-READING QUESTION

Open script.js. Find one function that supports adding the responsive navigation. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a JavaScript file
- control responsive navigation
- update accessibility attributes
- respond to user interactions
- coordinate HTML, CSS and JavaScript
- improve the mobile experience

In the next lesson, you will review the complete public website, improve its overall quality, and prepare it for the Supabase connection in Chapter 2.

CHAPTER 1

BUILDING THE PUBLIC WEBSITE

LESSON 5

COMPLETING THE PUBLIC WEBSITE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will review and improve the complete public website for your Appointment Booking System.

You have already built:

- index.html
- styles.css
- script.js

Your website is functional, but like most software, it can still be improved.

Rather than adding new features, you will refine what already exists while preserving every working part of the project.

WHY THIS MATTERS

Professional software is rarely perfect after the first version.

Developers continuously improve layouts, wording, accessibility, responsiveness, spacing, and usability.

These improvements make the software easier to understand and more pleasant to use without changing its purpose.

Learning how to improve existing software safely is one of the most valuable software development skills.

BEFORE YOU CONTINUE

Confirm that your Appointment Booking System folder contains:

index.html

styles.css

script.js

Open index.html in your browser.

Confirm that:

☒ The landing page opens.

☒ The page is fully styled.

☒ The responsive navigation works.

☒ The Login link points to:

login.html

☒ The Register link points to:

register.html

Do not continue until everything above works correctly.

BUILD PROMPT

PROMPT STARTS HERE

I already have a complete public website for my Appointment Booking System.

Everything already built must continue working.

Do not remove any existing functionality.

Return complete updated versions of every affected file.

Do not return snippets.

Do not return explanations.

Return complete updated versions of:

index.html

styles.css

script.js

only if improvements are required.

GENERAL GOAL

Review the complete public website and improve its overall quality while preserving every existing feature.

INDEX.HTML

Keep every existing section.

Do not remove:

- Header
- Navigation
- Hero
- Business Problem
- Features
- How It Works
- Dashboard Preview
- Final Call-to-Action
- Footer

Improve where appropriate:

- Heading wording
- Paragraph wording
- Accessibility labels
- Semantic HTML

Do not rename IDs unless absolutely necessary.

Keep:

```
<link rel="stylesheet" href="styles.css">
```

Keep:

```
<script src="script.js"></script>
```

Continue pointing:

Login

to:

login.html

Continue pointing:

Register

to:

register.html

STYLES.CSS

Keep every existing style.

Improve where appropriate:

- Consistent spacing
- Typography
- Button styling
- Card alignment
- Section spacing
- Mobile layout
- Dashboard preview appearance
- Footer layout

Improve keyboard focus styles where appropriate.

Maintain good colour contrast.

Do not redesign the page completely.

Refine the existing design.

SCRIPT.JS

Keep every existing feature working.

Review the JavaScript for:

- Duplicate event listeners
- Repeated logic
- Unnecessary code
- Accessibility improvements

Continue supporting:

- Responsive navigation
- Hamburger menu
- Navigation closing
- Escape key

- Outside click
 - Desktop resize handling
- Do not add authentication code.
- Do not add Supabase code.
- Do not book appointment management code.

QUALITY REVIEW

Review the complete website and ensure:

- Navigation feels smooth.
- Buttons are consistent.
- Messages and wording are professional.
- Layout spacing is balanced.
- Mobile experience is clean.
- Keyboard navigation still works.

IMPORTANT

- Everything already built must continue working.
- Do not remove features.
- Do not rename files.
- Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

- ChatGPT should return complete updated versions of any files that require improvement.
- Every returned file should be complete from beginning to end.
- Nothing that already works should stop working.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

If ChatGPT returns an updated version of any file:

Open that file in Notepad.

Cancel the existing contents.

Paste the complete updated code.

Click File.

Click Save.

Repeat this process for every updated file.

Do not create duplicate files.

Always replace the complete contents of the existing file.

Your project folder should still contain only:

index.html

styles.css

script.js

TEST YOUR WORK

Refresh:

index.html

Review the complete landing page.

Confirm that:

- ☒ The header still appears correctly.
- ☒ Navigation still works.
- ☒ The hamburger menu still works.
- ☒ Every section remains present.
- ☒ Buttons remain consistent.
- ☒ The dashboard preview still looks professional.
- ☒ The footer still appears correctly.

Now reduce the browser width.

Confirm that:

- ☒ The layout adjusts correctly.
- ☒ The hamburger menu still works.
- ☒ No content runs outside the page.
- ☒ No unnecessary horizontal scrolling appears.

Finally, click:

Home

Features

How It Works

Login

Register

Confirm that:

- ☒ Internal navigation still works.
- ☒ Login still points to login.html.
- ☒ Register still points to register.html.

CHECKPOINT

Before moving on, confirm that:

- ☒ The landing page looks professional.

- ✓ Every section remains present.
- ✓ Navigation still works.
- ✓ Responsive behaviour still works.
- ✓ Login and Register links remain correct.
- ✓ No existing functionality has been lost.

COMMON BEGINNER MISTAKES

Some beginners ask ChatGPT to improve only a small section of a file.

That often causes important code to disappear.

Always request the complete updated file.

Another common mistake is accepting improvements without testing the complete website afterwards.

Even small changes can accidentally affect responsive layouts or navigation.

Refresh the browser and test the whole page after every update.

BEHIND THE SCENES

As software projects become larger, developers rarely rebuild them from scratch.

Instead, they improve existing files while preserving the features that already work.

Throughout this workbook, you will continue following this same approach.

Each Build Prompt will protect everything already built while adding or refining only what is necessary.

THINK LIKE A SOFTWARE DESIGNER

Professional software evolves through continuous improvement.

Instead of asking,

"What can I build next?"

experienced developers often ask,

"What can I make better?"

Learning to improve existing software without breaking it is one of the habits that separates experienced developers from beginners.

CODE-READING QUESTION

Open index.html. Find the part of the page added for completing the public website. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an existing project
- improve a public website safely
- preserve existing functionality
- refine layouts and wording
- test a website after improvements

CHAPTER SUMMARY

Congratulations.

You have completed the public website for your Appointment Booking System.

Your application now has a professional landing page that explains the software clearly and guides visitors towards creating an account.

You also built a responsive navigation system that works across different screen sizes.

Most importantly, you have followed a clear professional workflow.

You built the application one reliable step at a time, testing each stage before moving forward.

CHAPTER MILESTONE

By the end of Chapter 1, you have successfully built:

- ✓ Professional landing page
- ✓ Responsive navigation
- ✓ Three-line hamburger menu
- ✓ Hero section
- ✓ Business problem section
- ✓ Features section
- ✓ How It Works section
- ✓ Dashboard preview
- ✓ Final call-to-action
- ✓ Footer
- ✓ Login link
- ✓ Register link
- ✓ Responsive layout
- ✓ HTML, CSS and JavaScript working together

Your project folder should now contain:

index.html

styles.css

script.js

TRANSITION TO CHAPTER 2

Your public website is now complete.

Visitors can learn about your software and navigate through the landing page.

The next step is to connect your application to Supabase.

In Chapter 2, you will create your Supabase project, build a shared connection file, test the connection, and prepare the application for the complete authentication system that follows.

Once that foundation is complete, you will begin building secure user accounts and protected appointment management features.

CHAPTER 2

CONNECTING TO SUPABASE

CHAPTER INTRODUCTION

Your public website is now complete.

Visitors can learn about your Appointment Booking System, understand how it works, and choose to register for an account.

The next step is to give your application a secure online backend.

That backend is Supabase.

Supabase will be responsible for:

- User authentication
- Appointment data storage
- Database security
- Protecting every user's appointments

During this chapter, you will connect your website to your own Supabase project.

Although visitors will not register until Chapter 3, everything required to support registration and appointment management will already be in place.

Think of this chapter as laying the foundation before constructing the building.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Create a Supabase project
- Locate the Project URL
- Locate the Publishable Key

- Create a shared connection file
- Build a connection test page
- Verify that your application communicates successfully with Supabase

By the end of the chapter, your project will be fully prepared for authentication and appointment management.

LESSON 1

CREATING YOUR SUPABASE PROJECT

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a new Supabase project specifically for your Appointment Booking System.

Each workbook should have its own Supabase project.

Keeping projects separate makes them easier to manage and prevents one application from accidentally affecting another.

WHY THIS MATTERS

Your Appointment Booking System needs a secure online backend.

Without one, users cannot:

- Create accounts
- Sign in
- Store appointments
- Retrieve appointments later

Supabase provides all of these services.

BEFORE YOU CONTINUE

Make sure you have:

- ☒ A working internet connection.
- ☒ A web browser.
- ☒ Your Appointment Booking System folder.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create your Supabase project.

Follow these steps.

1.

Open your web browser.

2.

Visit:

<https://supabase.com>

3.

Create a free account if you do not already have one.

4.

Sign in.

5.

Click:

New Project

6.

Choose your organisation.

7.

Enter a project name.

For example:

Appointment Booking System

8.

Create a strong database password.

Store it somewhere safe.

Do not share it with anyone.

Your application will not use this password directly.

9.

Choose the region closest to you.

10.

Click:

Create New Project

Wait while Supabase prepares the project.

This may take several minutes.

Do not continue until the project is ready.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your web browser.

SAVE YOUR FILES

No project files are created during this lesson.

TEST YOUR WORK

After the project has been created successfully:

Open the project dashboard.

Confirm that:

- ☒ The project opens.
- ☒ The project status shows that it is ready.

CHECKPOINT

Before moving on, confirm that:

- ☒ Your Supabase account exists.
- ☒ Your new project has been created.
- ☒ The project dashboard opens successfully.
- ☒ The project is ready.

COMMON BEGINNER MISTAKES

Some students close the browser before Supabase finishes creating the project.

Always wait until the project is fully ready.

Another common mistake is forgetting the database password.

Store it somewhere safe before leaving this page.

Remember that your website will not use this password.

Later lessons will use the Publishable Key instead.

BEHIND THE SCENES

When Supabase creates your project, it automatically prepares:

- A PostgreSQL database
- User authentication
- Storage services
- Security settings
- APIs

These services work together to support the software you are building.

THINK LIKE A SOFTWARE DESIGNER

Professional developers normally create a separate backend for each application.

Keeping projects separate reduces confusion, improves security, and makes future maintenance much easier.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a Supabase project
- prepare an online backend

- understand why every software project should have its own database

In the next lesson, you will locate the Project URL and Publishable Key that allow your Appointment Booking System to communicate securely with Supabase.

CHAPTER 2
CONNECTING TO SUPABASE**LESSON 2**

UNDERSTANDING THE PROJECT URL AND PUBLISHABLE KEY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will locate the two pieces of information your Appointment Booking System needs to communicate with Supabase.

These are:

- The Project URL
- The Publishable Key

In the next lesson, you will place these values inside a reusable connection file that every page in your application will use.

WHY THIS MATTERS

Your website needs to know where your Supabase project is located before it can communicate with it.

The Project URL identifies your Supabase project.

The Publishable Key allows your website to communicate with that project securely.

These two values will be used throughout this workbook whenever your application:

- Registers a user
- Signs in a user
- Loads appointments
- Saves appointments
- Updates appointments

- Cancels appointments

BEFORE YOU CONTINUE

Confirm that:

- ☒ Your Supabase project has finished creating.
- ☒ You are signed in to Supabase.
- ☒ Your project dashboard is open.

No files will be created during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, locate your Project URL and Publishable Key.

Follow these steps carefully.

1.

Open your Supabase project.

2.

From the left-hand menu, click:

Project Settings

3.

Click:

Data API

Locate:

Project URL

It will look similar to:

<https://your-project-id.supabase.co>

Copy the complete value.

Do not change it.

Next, locate:

Publishable Key

Copy the complete key.

Store both values somewhere safe.

You will use them in the next lesson.

IMPORTANT

Your Project URL must end with:

.supabase.co

Do not add:

/rest/v1/

The correct format is:

`https://your-project-id.supabase.co`

IMPORTANT

Use only the:

Publishable Key

Never use:

- Service Role Key
- Secret Key
- Database password

Only the Publishable Key belongs inside your project files.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside your Supabase dashboard.

SAVE YOUR FILES

No project files are created during this lesson.

Keep your Project URL and Publishable Key ready for the next lesson.

TEST YOUR WORK

Confirm that you have copied:

- ☒ Your Project URL
- ☒ Your Publishable Key

Check the Project URL carefully.

It should end with:

.supabase.co

There should be nothing after that.

CHECKPOINT

Before moving on, confirm that:

- ☒ You have located your Project URL.
- ☒ You have located your Publishable Key.
- ☒ You copied the Publishable Key.
- ☒ You did not copy a secret key.
- ☒ Your Project URL does not contain:

/rest/v1/

COMMON BEGINNER MISTAKES

One common mistake is copying the wrong key.

Supabase displays several different keys.

Only the Publishable Key should be used in this workbook.

Never use the Service Role Key.

Never use a Secret Key.

Those keys are intended for secure server environments and should never appear inside files that visitors can download through their web browser.

Another common mistake is copying the REST API address instead of the Project URL.

Always use the base Project URL ending with:

.supabase.co

BEHIND THE SCENES

Whenever your application communicates with Supabase, it sends requests to your Project URL.

Those requests include your Publishable Key.

Supabase uses this information to identify your project before processing the request.

Although the Publishable Key is visible inside your website, it does not give visitors unrestricted access to your data.

Authentication and Row Level Security will provide that protection later in this workbook.

THINK LIKE A SOFTWARE DESIGNER

Professional software separates public information from private information.

The Project URL and Publishable Key are intended to be used by browser-based applications.

Sensitive credentials remain on secure servers and are never exposed to visitors.

Understanding this difference is one of the foundations of building secure software.

WHAT YOU LEARNED

In this lesson you learned how to:

- locate the Project URL
- locate the Publishable Key
- understand the purpose of both values
- avoid using secret keys
- prepare your project for the shared Supabase connection

In the next lesson, you will create a reusable Supabase connection file that every page in your Appointment Booking System will use.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 3

CREATING YOUR SUPABASE CONNECTION FILE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create a reusable JavaScript file that connects your Appointment Booking System to your Supabase project.

Rather than writing the connection code on every page, you will create one shared file named:

`supabase-config.js`

Every page that needs to communicate with Supabase will use this file.

This keeps your project organised and makes future updates much easier.

WHY THIS MATTERS

As software grows, several pages often need to communicate with the same database.

Instead of copying the connection code into every JavaScript file, professional developers create one shared connection file.

Whenever another page needs Supabase, it simply loads this file.

This approach reduces mistakes, keeps the project easier to maintain, and ensures every page uses exactly the same connection.

BEFORE YOU CONTINUE

Before starting this lesson, confirm that:

- ☒ Your Supabase project has been created.

✓ You have copied your Project URL.

✓ You have copied your Publishable Key.

Your Appointment Booking System folder should currently contain:

index.html

styles.css

script.js

You are about to add a fourth file.

BUILD PROMPT

PROMPT STARTS HERE

Create one complete JavaScript file named:

supabase-config.js

Return only the complete file.

Do not return explanations.

Do not return snippets.

Do not return HTML.

Do not return CSS.

GENERAL REQUIREMENTS

Create a reusable Supabase connection file.

Use the official Supabase browser client.

Create the client using this format exactly:

```
const supabaseClient = window.supabase.createClient(  
  "https://your-project-id.supabase.co",  
  "YOUR_PUBLISHABLE_KEY"  
);
```

Replace the placeholder values with my actual values.

Project URL:

PASTE YOUR PROJECT URL HERE

Publishable Key:

PASTE YOUR PUBLISHABLE KEY HERE

REQUIREMENTS

- Create only one Supabase client.
- Store it in a constant named:
`supabaseClient`
- Do not use:
`/rest/v1/`
inside the Project URL.
- Do not use:
Service Role Key
- Do not use:
Secret Key
- Do not use:
Database password
- Include a short comment at the top of the file explaining that this file creates the shared Supabase connection used throughout the application.
- Return one complete JavaScript file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`supabase-config.js`

The file should:

- ☒ Create one shared Supabase client.
- ☒ Use my Project URL.

☒ Use my Publishable Key.

☒ Create the client using:

```
const supabaseClient = window.supabase.createClient(...)
```

☒ Contain JavaScript only.

SAVE YOUR FILES

Copy the complete JavaScript returned by ChatGPT.

Open Notepad.

Paste the code into a new empty document.

Click File.

Click Save As.

Open your Appointment Booking System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

supabase-config.js

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

Make sure the file is not named:

supabase-config.js.txt

TEST YOUR WORK

Open your Appointment Booking System folder.

Confirm that:

supabase-config.js

appears alongside the other project files.

Open the file in Notepad.

Confirm that:

☒ The Project URL ends with:

.supabase.co

☒ The Project URL does not contain:

/rest/v1/

☒ The Publishable Key appears.

☒ The Supabase client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

At this stage, the connection file is not yet being used by any webpage.

That is expected.

The connection will be tested in the next lesson.

CHECKPOINT

Before moving on, confirm that:

☒ supabase-config.js has been created.

☒ The filename is correct.

☒ The Project URL is correct.

☒ The Publishable Key is correct.

☒ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

One common mistake is copying the REST API address instead of the Project URL.

Always use the base URL ending with:

`.supabase.co`

Another common mistake is copying the Service Role Key instead of the Publishable Key.

Only the Publishable Key belongs inside this file.

Never place secret keys inside browser-based applications.

Some beginners also create another Supabase client later inside `register.js` or `dashboard.js`.

Do not do that.

Throughout this workbook, every page will use the single shared client created inside:

`supabase-config.js`

BEHIND THE SCENES

The line:

```
const supabaseClient = window.supabase.createClient(...)
```

creates one reusable connection between your application and your Supabase project.

Later in this workbook, every page that needs Supabase will simply use:

`supabaseClient`

instead of creating another connection.

This keeps the application consistent and greatly reduces the chance of connection errors.

THINK LIKE A SOFTWARE DESIGNER

Whenever several parts of an application need the same functionality, it is usually better to create that functionality once and reuse it everywhere.

This principle keeps software simpler, easier to maintain, and less likely to contain conflicting code.

Creating a shared Supabase connection file is one example of that idea.

CODE-READING QUESTION

Open supabase-config.js. Which line creates the Supabase client, and which two saved values does that line use?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a shared Supabase connection file
- create one reusable Supabase client
- use the correct Project URL
- use the correct Publishable Key
- avoid exposing secret keys
- prepare your application for future database communication

In the next lesson, you will build a connection test page that confirms your Appointment Booking System can communicate successfully with your Supabase project.

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 4

BUILDING A CONNECTION TEST PAGE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a simple webpage that confirms your Appointment Booking System can communicate successfully with your Supabase project.

This page exists only for testing.

It is not part of the finished application.

Instead, it allows you to verify that your shared Supabase connection is working correctly before you begin building authentication.

WHY THIS MATTERS

Professional developers test important parts of an application before building additional features.

Imagine spending hours creating registration and login pages only to discover that your website was never connected to Supabase correctly.

Testing the connection now helps you identify problems while the project is still simple.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

You should also have:

- ☒ Your Supabase project
- ☒ Your Project URL
- ☒ Your Publishable Key

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have these files:

index.html

styles.css

script.js

supabase-config.js

Everything already built must continue working.

Do not modify any existing files.

Create one new HTML file named:

test-connection.html

Return one complete HTML file only.

Do not return explanations.

Do not return snippets.

GENERAL REQUIREMENTS

Create a complete HTML5 document.

Include:

- Appropriate page title
- Viewport meta tag

Inside the head section, include simple CSS that creates a clean, centred layout suitable for a testing page.

BODY CONTENT

Display:

Heading

Supabase Connection Test

Below the heading, display a short paragraph explaining that the page is checking communication with the Supabase project.

Create a status box.

The status box should initially display:

Checking connection...

Give this element the ID:

connection-status

SCRIPT LOADING ORDER

Immediately before the closing body tag, load these scripts in this exact order.

First:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

Second:

```
<script src="supabase-config.js"></script>
```

Third:

Create an inline JavaScript block that:

- Checks whether:

supabaseClient

exists.

If it exists:

Change the status box to display:

✅ Connected successfully to Supabase.

Apply a success style.

If it does not exist:

Display:

❌ Unable to connect to Supabase.

Apply an error style.

IMPORTANT

Use the existing:

`supabase-config.js`

Do not create another Supabase client.

Do not hard-code the Project URL.

Do not hard-code the Publishable Key.

Do not use authentication.

Do not create database tables.

Do not create users.

Return one complete HTML file only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return one complete file:

`test-connection.html`

The page should:

- ☒ Load the Supabase browser library.
- ☒ Load `supabase-config.js`.
- ☒ Load scripts in the correct order.
- ☒ Check whether `supabaseClient` exists.
- ☒ Display a clear success or error message.

SAVE YOUR FILES

Copy the complete HTML returned by ChatGPT.

Open Notepad.

Paste the code into a new document.

Click File.

Click Save As.

Open your Appointment Booking System folder.

Choose:

Save as type:

All Files

Enter the filename exactly as:

test-connection.html

Click Save.

Close Notepad.

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TEST YOUR WORK

Open:

test-connection.html

The page should open in your browser.

After a moment, the message should change from:

Checking connection...

to:

✅ Connected successfully to Supabase.

If instead you see:

❌ Unable to connect to Supabase.

Do not continue.

Return to the previous lessons and carefully check:

- ✓ Project URL
- ✓ Publishable Key
- ✓ supabase-config.js
- ✓ Script loading order

CHECKPOINT

Before moving on, confirm that:

- ✓ test-connection.html opens.
- ✓ The page loads correctly.
- ✓ The Supabase browser library loads first.
- ✓ supabase-config.js loads second.
- ✓ The page displays:

Connected successfully to Supabase.

COMMON BEGINNER MISTAKES

The most common mistake is loading the JavaScript files in the wrong order.

The correct order is always:

1. Supabase browser library
2. supabase-config.js
3. Page-specific JavaScript

Another common mistake is creating another Supabase client inside the page.

Do not do this.

Your application should use only the shared client created inside:

supabase-config.js

BEHIND THE SCENES

This page does not create a new connection.

Instead, it loads the shared connection file.

If that file creates:

`supabaseClient`

successfully, the page simply confirms that the shared connection is available.

Every protected page you build later will use this same approach.

THINK LIKE A SOFTWARE DESIGNER

Professional software is built on reliable foundations.

Testing one important component before moving to the next makes larger projects much easier to debug.

Finding problems early almost always saves time later.

CODE-READING QUESTION

Open `test-connection.html`. Find the part of the page added for building a connection test page. Which HTML element starts that part?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a dedicated connection test page
- load the Supabase browser library
- reuse a shared connection file
- verify that your application can communicate with Supabase
- identify common connection problems

CHAPTER 2

CONNECTING TO SUPABASE

LESSON 5

FINAL CONNECTION REVIEW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

This lesson is your final review before building the authentication system.

You are not creating any new files.

Instead, you will carefully inspect your project and confirm that every connection to Supabase has been configured correctly.

WHY THIS MATTERS

The authentication system depends completely on your Supabase connection.

If the connection is incorrect, registration, login, appointment management and every future database feature will fail.

Confirming your work now helps prevent much larger problems later.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, review your project carefully.

Complete the following checks.

Open:

supabase-config.js

Confirm that:

☒ The Project URL ends with:

.supabase.co

☒ The Project URL does not contain:

/rest/v1/

☒ The Publishable Key is being used.

☒ The shared client is created using:

```
const supabaseClient = window.supabase.createClient(...)
```

Next, open:

test-connection.html

Confirm that the scripts load in this exact order:

1.

Supabase browser library

2.

supabase-config.js

3.

The page's own JavaScript

Finally, open:

test-connection.html

Confirm that the page displays:

☒ Connected successfully to Supabase.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed by reviewing your project.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this checklist.

Landing Page

☒ index.html opens correctly.

☒ Navigation works.

☒ Responsive menu works.

Supabase

☒ Project created.

☒ Project URL correct.

☒ Publishable Key correct.

☒ Shared connection file created.

Connection Test

☒ test-connection.html opens.

☒ Connected successfully to Supabase appears.

Project Folder

☒ index.html

☒ styles.css

☒ script.js

✓ supabase-config.js

✓ test-connection.html

CHECKPOINT

Before moving to Chapter 3, confirm that:

✓ Every required file exists.

✓ The landing page still works.

✓ The connection test succeeds.

✓ The Project URL is correct.

✓ The Publishable Key is correct.

✓ The script loading order is correct.

✓ Only one Supabase client exists.

COMMON BEGINNER MISTAKES

Some students continue building after the connection test has failed.

Do not do this.

Authentication depends on the connection working correctly.

Solve connection problems now before moving to the next chapter.

BEHIND THE SCENES

At this point, your Appointment Booking System knows how to communicate with Supabase.

What it cannot do yet is identify users.

That is the purpose of the next chapter.

Authentication allows the application to recognise who is currently signed in so every appointment can be linked to the correct person.

THINK LIKE A SOFTWARE DESIGNER

Every large software project is built in layers.

Public website.

Backend connection.

Authentication.

Database.

Business features.

Building each layer carefully creates a much more reliable application than trying to build everything at once.

WHAT YOU LEARNED

In this lesson you learned how to:

- verify the shared Supabase connection
- review script loading order
- confirm that your application is ready for authentication
- understand why authentication comes before appointment management

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System is now successfully connected to Supabase.

You created your own Supabase project, located the Project URL and Publishable Key, built a shared connection file, and confirmed that your application can communicate successfully with your online backend.

More importantly, you established a reusable architecture that every future page in the application will use.

Whenever a page needs Supabase, it will always load:

1. The Supabase browser library.
2. `supabase-config.js`.
3. The page-specific JavaScript file.

Maintaining this structure throughout the project will keep your application organised and reduce connection-related problems.

CHAPTER MILESTONE

By the end of Chapter 2, you have successfully completed:

- ✓ Supabase project
- ✓ Project URL
- ✓ Publishable Key
- ✓ Shared connection file
- ✓ Connection test page
- ✓ Connection verification
- ✓ Correct script loading order
- ✓ Backend preparation for authentication

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

TRANSITION TO CHAPTER 3

Your public website is complete.

Your application is connected to Supabase.

The next step is to build the complete authentication system.

In Chapter 3, you will build registration, login, the protected dashboard, and the protected appointment details page as one coordinated system.

By creating every destination page together, every redirect will already have a valid destination, allowing the complete authentication workflow to function correctly from the very beginning.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

CHAPTER INTRODUCTION

Your Appointment Booking System now has a professional public website and a working connection to Supabase.

The next step is to allow people to create their own accounts and access their own appointments securely.

Authentication is one of the most important parts of any business application.

Without authentication, every visitor would have access to the same appointment database.

That would make the software unsafe to use.

In this chapter, you will build the complete authentication system as one coordinated feature.

Instead of creating one page at a time, you will create every authentication page together.

This approach ensures that:

- Every redirect already has a destination.
- Every page is correctly linked.
- Every JavaScript file loads correctly.
- Every stylesheet is linked correctly.
- The complete user journey works from the very beginning.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- register.html
- login.html

- dashboard.html
- appointment-details.html
- auth.css
- register.js
- login.js
- dashboard.js
- appointment-details.js

You will also update:

- index.html

The completed authentication flow will work like this:

Visitor

↓

index.html

↓

Register

↓

register.html

↓

Email verification

↓

login.html

↓

Login

↓

dashboard.html

↓

Appointment Details

↓

appointment-details.html

↓

Logout

↓

login.html

Unauthenticated visitors who attempt to open:

dashboard.html

or

appointment-details.html

will be redirected automatically to:

login.html

LESSON 1

UNDERSTANDING THE AUTHENTICATION FLOW

Estimated Time

15 minutes

WHAT YOU ARE BUILDING

Before creating the authentication system, it is important to understand how every page works together.

Authentication is not simply a login page.

It is a complete journey that begins when a visitor creates an account and continues until they safely sign out.

Every page depends on another page.

That is why the entire system will be built together.

WHY THIS MATTERS

Many beginners build authentication one page at a time.

The result is often broken redirects, missing files and incomplete user journeys.

Professional developers think about the complete workflow before writing any code.

Doing so reduces mistakes and produces a much more reliable application.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

- ✓ The landing page works.
- ✓ The Supabase connection test succeeds.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, study the authentication flow below.

A visitor arrives at:

`index.html`

The visitor selects:

Register

The visitor opens:

`register.html`

The visitor creates an account.

Supabase sends a verification email.

The visitor verifies the email address.

The visitor returns to:

`login.html`

The visitor signs in.

The application redirects to:

`dashboard.html`

The user manages appointments.

Whenever the user opens an appointment details, the application opens:

`appointment-details.html`

When the user signs out, the application redirects to:

`login.html`

If the visitor is not authenticated and tries to open:

`dashboard.html`

or

`appointment-details.html`

the application immediately redirects them to:

```
login.html
```

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

Review the authentication flow.

Make sure you understand where every page sends the user.

Every destination page already has a purpose.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Registration comes before Login.
- ☒ Login comes before Dashboard.
- ☒ Appointment Details is protected.
- ☒ Logout returns the user to login.html.
- ☒ Protected pages require authentication.

COMMON BEGINNER MISTAKES

One common mistake is thinking that a login page alone creates a complete authentication system.

A secure authentication system includes:

- Registration
- Email verification

- Login
- Protected pages
- Logout
- Automatic redirects

All of these parts work together.

BEHIND THE SCENES

Authentication allows the application to answer one important question:

Who is currently using the software?

Once that question has been answered, every appointment can be connected to the correct user.

THINK LIKE A SOFTWARE DESIGNER

Always think about the complete journey.

Instead of asking:

"What happens on this page?"

ask:

"Where did the user come from?"

and

"Where should the user go next?"

That way, every page becomes part of one connected application.

WHAT YOU LEARNED

In this lesson you learned:

- the complete authentication workflow
- how every authentication page connects together
- why protected pages exist
- why redirects are planned before building begins

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

Estimated Time

75 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete authentication system for your Appointment Booking System.

Rather than generating one page at a time, ChatGPT will generate every authentication page together.

This ensures that:

- Every destination page already exists.
- Every redirect already has somewhere to go.
- Every stylesheet is linked correctly.
- Every JavaScript file is loaded correctly.
- Everything already built continues working.

WHY THIS MATTERS

Registration, login, dashboard access and appointment details are tightly connected.

Building them together reduces missing files, broken redirects and inconsistent behaviour.

This careful approach reduces missing files and broken redirects.

BEFORE YOU CONTINUE

Your project folder should currently contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

Confirm that:

☒ The connection test displays:

Connected successfully to Supabase.

Do not continue until everything above is working.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Appointment Booking System.

Everything already built must continue working.

My project currently contains:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

The public website is complete.

The responsive navigation works.

The Supabase connection has been tested successfully.

Do not remove or break any existing functionality.

Return complete files only.

Do not return snippets.

Do not return explanations.

Return the following complete files:

1.

Updated index.html

2.

register.html

3.

login.html

4.

dashboard.html

5.

appointment-details.html

6.

auth.css

7.

register.js

8.

login.js

9.

dashboard.js

10.

appointment-details.js

GENERAL REQUIREMENTS

Everything already built must continue working.

Every HTML page must be complete.

Every HTML page must include proper HTML5 structure.

Every HTML page must include an appropriate page title.

Use semantic HTML.

Use modern, responsive layouts.

The logo on every page should display:

Appointment Records

The logo should link to:

index.html

INDEX.HTML

Return a complete updated version.

Preserve:

- Hero
- Features
- Navigation
- Footer
- Responsive menu

Ensure Login points to:

login.html

Ensure Register points to:

register.html

REGISTER.HTML

Create a professional registration page.

Include:

- Logo
- Client Name
- Email Address
- Password
- Confirm Password
- Register button
- Link to login.html

Display friendly validation messages.

Disable the Register button while processing.

Display loading messages.

After successful registration:

Display a success message explaining that the user must verify their email before signing in.

Redirect the user to:

login.html

Do not automatically sign the user in.

LOGIN.HTML

Create a professional login page.

Include:

- Logo
- Email
- Password
- Login button
- Link to register.html

Display friendly validation and loading messages.

Disable the Login button while processing.

If login succeeds:

Redirect to:

dashboard.html

If the user is already authenticated:

Automatically redirect to:

dashboard.html

DASHBOARD.HTML

Create a professional dashboard page.

Include:

- Logo
- Welcome heading
- Logged-in user's email
- Four appointment summary cards with temporary values:

Total Appointments

Today's Appointments

Upcoming Appointments

Cancelled Appointments

Include a placeholder section where the appointment management dashboard will be built later.

Include:

Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

APPOINTMENT-DETAILS.HTML

Create a protected appointment details page.

Include:

- Logo
- Appointment details heading
- Placeholder appointment information
- Back to Dashboard button
- Logout button

If the visitor is not authenticated:

Redirect immediately to:

login.html

AUTH.CSS

Create one shared stylesheet for:

register.html

login.html

dashboard.html

appointment-details.html

Use:

Professional forms

Responsive layout

Business dashboard styling

Summary cards

Loading messages

Success messages

Error messages

REGISTER.JS

Load after:

1. Supabase CDN
2. supabase-config.js

Use:

```
const supabaseClient
```

already created inside:

```
supabase-config.js
```

Do not create another Supabase client.

Validate:

- Client Name
- Email
- Password
- Confirm Password

Passwords must match.

Create the account using Supabase Authentication.

When calling:

```
supabaseClient.auth.signUp
```

include an email confirmation destination.

Create it from the website that is currently open:

```
`${window.location.origin}/login.html`
```

Pass that address as:

```
emailRedirectTo
```

inside the sign-up options.

Do not hardcode a Netlify website name.

Using:

```
window.location.origin
```

allows the same complete files to work when the website address changes.

Display friendly success and error messages.

Disable the button while processing.

After successful registration:

Inform the user to open the verification email.

Redirect to:

login.html

Do not automatically sign the user in from the registration form.

LOGIN.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Authenticate users.

Display loading messages.

Display friendly errors.

Redirect successful logins to:

dashboard.html

DASHBOARD.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the user is authenticated.

Display the authenticated user's email.

Support Logout.

Redirect unauthenticated users to:

login.html

APPOINTMENT-DETAILS.JS

Load after:

1. Supabase CDN

2. supabase-config.js

Use:

supabaseClient

Confirm that the visitor is authenticated.

If not:

Redirect immediately to:

login.html

Support Logout.

Appointments will be loaded later in this workbook.

For now, create the protected page and its authentication logic.

SCRIPT LOADING ORDER

Every Supabase-powered HTML page must load:

1.

Supabase browser library

2.

supabase-config.js

3.

Its own page-specific JavaScript

IMPORTANT

Return every requested file completely.

Do not omit any file.

Do not return partial updates.

Everything already built must continue working.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return ten complete files.

Every HTML page should already contain the correct stylesheet and JavaScript links.

Every protected page should already verify authentication.

Every redirect should already point to an existing page.

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 2 (CONTINUED)

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

ChatGPT should return ten complete files.

Work through them one at a time.

If the file already exists, replace its complete contents.

If the file does not already exist, create it.

Do not copy only part of a file.

Always replace the complete contents.

Updated index.html

Open:

index.html

in Notepad.

Select all the existing code.

Cancel it.

Paste the complete updated code returned by ChatGPT.

Click:

File

Save

Close Notepad.

register.html

Open Notepad.

Create a new document.

Paste the complete code.

Click:

File

Save As

Open your Appointment Booking System folder.

Choose:

Save as type:

All Files

Enter:

register.html

Click Save.

login.html

Repeat the same process.

Save as:

login.html

dashboard.html

Repeat the same process.

Save as:

dashboard.html

appointment-details.html

Repeat the same process.

Save as:

appointment-details.html

auth.css

Repeat the same process.

Save as:

auth.css

register.js

Repeat the same process.

Save as:

register.js

login.js

Repeat the same process.

Save as:

login.js

dashboard.js

Repeat the same process.

Save as:

dashboard.js

appointment-details.js

Repeat the same process.

Save as:

appointment-details.js

When you finish, your project folder should contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

appointment-details.html

auth.css

register.js

login.js

dashboard.js

appointment-details.js

Check every filename carefully.

Do not allow any file to become:

register.html.txt

login.js.txt

dashboard.html.txt

or any similar variation.

TEST YOUR WORK

Do not assume the authentication system works.

Authentication must be tested from a website with an HTTPS address.

Do not test this chapter by double-clicking the HTML files in your project folder.

The files still remain inside your project folder.

You will simply place a test copy of the complete folder online before testing.

BEFORE YOU START THE TESTS

1.

Save every updated file in Notepad.

2.

Deploy the complete project folder to Netlify.

If you already created a Netlify test website for this project, deploy the updated folder to the same website.

3.

Copy the website address supplied by Netlify.

It should begin with:

`https://`

For example:

`https://your-site-name.netlify.app`

Your own website address will be different.

4.

Open your Supabase project.

Go to:

Authentication

Then open:

URL Configuration

5.

Set the Site URL to your Netlify website address.

Do not add a file name to the Site URL.

6.

Add this complete address to Redirect URLs:

`https://your-site-name.netlify.app/login.html`

Replace the example website name with your real Netlify website name.

7.

Make sure the latest version of every file has been deployed.

8.

Open the project from its Netlify HTTPS address.

Keep using that address throughout every test below.

TEST 1

PUBLIC WEBSITE

Open the deployed index.html page.

Confirm that:

- ☒ The landing page opens correctly.
- ☒ The page is fully styled.
- ☒ The responsive navigation still works.
- ☒ Login opens the deployed login.html page.
- ☒ Register opens the deployed register.html page.

Look at the browser address bar.

The address must begin with:

https://

Nothing already built should have stopped working.

TEST 2

REGISTRATION PAGE

Open the deployed register.html page.

Attempt to submit the form without entering any information.

Friendly validation messages should appear.

Complete the form using an email address that has not already been used for this test.

Click:

Register

The Register button should become disabled while the request is being processed.

A loading message should appear.

After successful registration:

You should see a message asking you to verify your email.

TEST 3

EMAIL VERIFICATION

Open your email inbox.

Locate the verification email from Supabase.

Open the email.

Click the verification link.

Supabase should verify the email and return the browser to your deployed website.

The returned address should use your Netlify website name.

It should not begin with:

file://

Depending on the authentication response, you may briefly see login.html or the application may recognise the new session and open the dashboard.

Both results confirm that the browser returned to the correct website.

If the login page remains open, sign in using the email address and password you registered.

If the verification email does not arrive immediately:

Wait a few minutes.

Refresh your inbox.

Check your Spam or Junk folder.

If the verification link opens the wrong website:

Return to Supabase URL Configuration.

Check the Site URL and Redirect URLs carefully.

Correct them before creating another test account.

TEST 4

LOGIN

Open the deployed login.html page.

Enter an incorrect password.

A friendly error message should appear.

Now enter the correct password.

Click:

Login

The Login button should become disabled while signing in.

A loading message should appear.

After a successful login:

The browser should open the deployed dashboard.html page.

TEST 5

AUTHENTICATED SESSION

After signing in successfully:

Confirm that the protected dashboard opens.

Confirm that your email address appears.

Refresh the page.

The dashboard should remain open because the same website still has your authenticated session.

Confirm that the four appointment summary cards appear.

Confirm that the Logout button appears.

While signed in, open the complete deployed appointment-details.html address.

The protected appointment details page should open.

Click:

Logout

The browser should return to the deployed login.html page.

While signed out, open the complete deployed dashboard.html address.

The application should immediately return you to the deployed login.html page.

While still signed out, open the complete deployed appointment-details.html address.

The application should immediately return you to the deployed login.html page.

Sign in again.

While signed in, open the complete deployed login.html address.

The application should immediately return you to the deployed dashboard.html page.

CHECKPOINT

Before moving to the next lesson, confirm that:

- ☒ Registration works.
- ☒ Validation works.
- ☒ Loading messages appear.
- ☒ Email verification succeeds.
- ☒ Login works.
- ☒ Friendly error messages appear.
- ☒ Dashboard is protected.
- ☒ Appointment Details page is protected.
- ☒ Logout works.
- ☒ Authenticated users cannot remain on login.html.
- ☒ Unauthenticated users cannot open dashboard.html.
- ☒ Unauthenticated users cannot open appointment-details.html.

COMMON BEGINNER MISTAKES

One common mistake is forgetting to verify the email address before attempting to sign in.

If email verification is enabled, Supabase will not allow the account to be used until verification has been completed.

Another common mistake is loading the JavaScript files in the wrong order.

Every Supabase-powered page should load:

1.

Supabase browser library

2.

supabase-config.js

3.

The page-specific JavaScript

Changing this order can prevent the application from working correctly.

Some beginners also create another Supabase client inside:

register.js

login.js

dashboard.js

or

appointment-details.js

Do not do this.

Every page should use the shared client created inside:

supabase-config.js

BEHIND THE SCENES

Your authentication system is now working as one connected application.

When a visitor creates an account, Supabase stores the authentication information securely.

After email verification, Supabase allows that account to sign in.

When login succeeds, Supabase creates a session.

Protected pages check whether that session exists before displaying their content.

If the session does not exist, the visitor is redirected immediately to the login page.

The Appointment Details page already uses this protection even though appointments have not yet been introduced.

Later, this same authentication system will protect every appointment in the application.

THINK LIKE A SOFTWARE DESIGNER

Authentication is more than allowing someone to sign in.

It controls who can enter the application, which pages they can open, and what information they are allowed to see.

Every protected page should perform its own authentication check.

Never assume that because a user reached one protected page, every other page is automatically secure.

CODE-READING QUESTION

Open `register.js`. Find one function that supports building the complete authentication system. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete authentication system
- connect registration, login and protected pages
- protect multiple destinations
- redirect unauthenticated visitors
- redirect authenticated users appropriately
- preserve every feature already built

CHAPTER 3

BUILDING THE COMPLETE AUTHENTICATION SYSTEM

LESSON 3

AUDITING THE AUTHENTICATION SYSTEM

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will not build new features.

Instead, you will carefully review every part of the authentication system before beginning appointment management.

Authentication is the foundation of the entire application.

Every appointment created later will depend on the current user being identified correctly.

WHY THIS MATTERS

The next chapter introduces appointments.

Every appointment must belong to one authenticated user.

If authentication is unreliable, appointment privacy cannot be guaranteed.

Professional developers verify security before adding business features.

BEFORE YOU CONTINUE

Your project folder should now contain:

index.html

styles.css

script.js

supabase-config.js

test-connection.html

register.html

login.html

dashboard.html

appointment-details.html

auth.css

register.js

login.js

dashboard.js

appointment-details.js

Confirm that you have successfully created at least one verified user account.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every authentication test again.

Do not continue until every test succeeds.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete this final authentication check on the deployed HTTPS website.

Do not complete this audit from files opened with:

file://

SUPABASE URL SETTINGS

- ✓ The Site URL contains the Netlify website address.
- ✓ Redirect URLs contains the complete deployed login.html address.

REGISTRATION AND VERIFICATION

- ✓ Registration works from the deployed register.html page.
- ✓ A verification email arrives.
- ✓ The verification link returns to the deployed website.
- ✓ The returned address begins with https:// and uses the correct Netlify website name.

LOGIN AND SESSION

- ✓ Login works from the deployed login.html page.
- ✓ Refreshing a protected page keeps a signed-in user signed in.
- ✓ Logout works.

PROTECTION

- ✓ A signed-out visitor cannot remain on dashboard.html.
- ✓ A signed-in user can reach dashboard.html.
- ✓ A signed-in user who opens login.html returns to dashboard.html.

PROTECTED APPOINTMENT PAGE

- ✓ A signed-in user can open appointment-details.html.
- ✓ A signed-out visitor cannot remain on appointment-details.html.

PAGES

- ✓ The signed-in user's email appears.
- ✓ The summary cards appear.

- ✓ The appointment details placeholder appears.

If every check passes, the authentication system is ready for the next chapter.

CHECKPOINT

Before moving to Chapter 4, confirm that:

- ✓ Every authentication page works.
- ✓ Every redirect works.
- ✓ Dashboard protection works.
- ✓ Appointment Details protection works.
- ✓ No authentication errors remain.

COMMON BEGINNER MISTAKES

Some students test only the successful login process.

Always test unsuccessful situations as well.

Protected pages should remain protected when no user is signed in.

Another common mistake is assuming one protected page automatically secures every other page.

Each protected page should verify authentication independently.

BEHIND THE SCENES

The authentication system now provides the identity that every future database operation will rely on.

When appointments are introduced in Chapter 4, every new appointment will automatically belong to the currently authenticated user.

Authentication and Row Level Security will work together to protect appointment information.

THINK LIKE A SOFTWARE DESIGNER

Security should be confirmed before adding business functionality.

A reliable authentication system makes every future feature easier to build because the application always knows who the current user is.

WHAT YOU LEARNED

In this lesson you learned how to:

- audit an authentication system
- verify protected pages
- verify redirects
- prepare the application for secure appointment management

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System now includes a complete authentication system.

Visitors can register, verify their client email addresses, sign in, access protected pages, and sign out securely.

Every protected page already knows how to verify the current user's session before displaying information.

Most importantly, your application can now identify who is currently using the system.

That capability will allow every appointment to belong to the correct user.

CHAPTER MILESTONE

By the end of Chapter 3, you have successfully built:

- ✓ User registration
- ✓ Email verification
- ✓ Secure login
- ✓ Protected dashboard
- ✓ Protected appointment details page
- ✓ Logout
- ✓ Friendly validation
- ✓ Friendly success messages
- ✓ Friendly error messages
- ✓ Loading states
- ✓ Shared authentication styling
- ✓ Complete authentication workflow

TRANSITION TO CHAPTER 4

Your Appointment Booking System can now identify every authenticated user.

The next step is to create the secure appointment database.

In Chapter 4, you will design the appointments table, create a richer business data model, enable Row Level Security, and prepare the application to store appointment information safely.

This workbook introduces a complete database structure for managing appointment details safely.

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

CHAPTER INTRODUCTION

Your Appointment Booking System can now recognise who is signed in.

The next step is to create a secure database where each authenticated user can store and manage their own appointments.

Every appointment needs enough information to answer five simple questions:

- Who is the appointment for?
- What service is being booked?
- When does it begin?
- When does it end?
- What is its current status?

You will also prevent two active appointments belonging to the same user from using overlapping times.

Row Level Security will make sure one user can never read or change another user's appointments.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- appointments table
- Appointment date and time fields
- Required and optional fields
- Allowed appointment statuses
- Protection against invalid time ranges

- Protection against overlapping active appointments
- User ownership
- Row Level Security
- SELECT, INSERT and UPDATE policies
- Secure cancellation support
- Two-account database testing

By the end of this chapter, your application will have a secure appointment database.

LESSON 1

UNDERSTANDING THE APPOINTMENT DATA MODEL

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

Before creating the database, you will learn what one appointment must contain.

Each appointment will store:

- Client name — required
- Client email — optional
- Client phone — optional
- Service — required
- Start date and time — required
- End date and time — required
- Status — booked, confirmed, completed or cancelled
- Notes — optional
- User ID — added automatically
- Created and updated times — added automatically

The start and end values will be stored as complete dates and times. This makes schedules easier to sort and display correctly.

WHY THIS MATTERS

Good software begins with good data design.

If the database is designed poorly, the software becomes difficult to improve later.

Professional developers spend time deciding what information should be stored before creating the database itself.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Registration works.
- ☒ Login works.
- ☒ Dashboard protection works.
- ☒ Your Supabase project is open.

No changes will be made to your project files during this lesson.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with AI in this lesson.

Study the appointment data model carefully.

Notice that client contact details are optional, but the client name, service, start time, end time and status are required.

Also notice that cancellation changes the appointment status. It does not permanently remove the record.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created during this lesson.

TEST YOUR WORK

Review the complete appointment data model.

Ask yourself:

Which fields are required?

Which fields are optional?

Why must the end of an appointment be later than its start?

If you can answer these questions, you are ready for the next lesson.

CHECKPOINT

Before moving on, confirm that you understand:

- ☒ Required fields
- ☒ Optional fields
- ☒ Why appointments contain richer information than simple records

COMMON BEGINNER MISTAKES

Some beginners make every field required.

This often frustrates users because they may not know every piece of information when creating a new appointment.

Another common mistake is collecting too little information.

An appointment database should contain enough detail to make future communication easier.

BEHIND THE SCENES

This database has been designed to balance simplicity with practical business use.

It collects enough information to support appointment management without becoming unnecessarily complicated.

Later workbooks will build on these same design principles.

THINK LIKE A SOFTWARE DESIGNER

Every database field should have a purpose.

Before adding a new field, ask yourself:

"Will this information help the business make better decisions or provide better service?"

If the answer is no, the field may not belong in the database.

WHAT YOU LEARNED

In this lesson you learned:

- how appointment information is organised
- the difference between required and optional fields
- why database design matters
- how business software stores richer information

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 2

CREATING THE APPOINTMENTS TABLE

Estimated Time

35 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the database table that stores every appointment.

Each row inside the table will represent one appointment.

Every appointment will belong to one authenticated user.

Later, the dashboard will use this table to create, display, update and cancel appointments.

WHY THIS MATTERS

Without a properly designed table, your application has nowhere to store appointment information.

Building the correct structure now makes every later feature much easier to develop.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Authentication is working.
- ☒ Your Supabase project is open.
- ☒ You understand the appointment data model.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I am building a beginner-friendly Appointment Booking System using HTML, CSS, Vanilla JavaScript, Supabase and Notepad.

Create one complete SQL file named:

appointments-table.sql

Return the complete file only.

Do not return a snippet or partial SQL.

The SQL must create a public.appointments table with:

- id – uuid primary key with gen_random_uuid()
- user_id – uuid required, referencing auth.users(id) with on delete cascade
- client_name – text required
- client_email – text optional
- client_phone – text optional
- service – text required
- starts_at – timestamp with time zone required
- ends_at – timestamp with time zone required
- status – text required with default booked
- notes – text optional
- created_at – timestamp with time zone required with default now()
- updated_at – timestamp with time zone required with default now()

Enable Row Level Security.

Do not create permissive policies.

Do not disable Row Level Security.

Add clear SQL comments that explain each section in simple language.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return one complete file:

- appointments-table.sql

The file should create the appointments table and enable Row Level Security.

SAVE YOUR FILES

Open Notepad.

Paste the complete SQL returned by AI.

Save the file inside your Appointment Booking System project folder as:

appointments-table.sql

In the Save As window, choose All Files so Notepad does not add .txt.

Confirm that the filename ends with .sql.

TEST YOUR WORK

Open appointments-table.sql in Notepad.

Copy the complete SQL.

Open the Supabase SQL Editor.

Paste and run the SQL once.

Then open Table Editor and confirm that:

- ☒ appointments exists.
- ☒ id is the Primary Key.
- ☒ user_id references auth.users.
- ☒ client_name and service are required.
- ☒ starts_at and ends_at use timestamp with time zone.
- ☒ status defaults to booked.

- ✓ Row Level Security is enabled.

Do not add a row manually yet.

CHECKPOINT

Before moving on, confirm that:

- ✓ The appointments table exists.
- ✓ Every required column exists.
- ✓ Every optional column exists.
- ✓ The Primary Key is correct.
- ✓ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

One common mistake is making the email field required.

In many real businesses, an appointment may not have an email address available.

Phone remains the primary contact method in this workbook.

Another common mistake is forgetting the `updated_at` column.

Although it is not immediately required, it becomes useful later when records are edited and sorted.

BEHIND THE SCENES

Every row in this table represents one complete appointment.

As the application grows, the software will retrieve, display and manage these appointments in different ways.

The structure you create today will support every feature built later.

THINK LIKE A SOFTWARE DESIGNER

A well-designed table should continue serving the application even after many new features are added.

Good database design anticipates future growth instead of solving only today's problem.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a richer database table
- distinguish required and optional fields
- prepare an appointment database for future features
- keep Row Level Security enabled

In the next lesson, you will learn how to use database constraints to improve data quality before creating the Row Level Security policies.

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 3

PROTECTING APPOINTMENT DATES, TIMES AND STATUSES

Estimated Time

25 minutes

WHAT YOU ARE BUILDING

In this lesson, you will add database rules that protect appointment quality.

The rules will reject blank names and services, unsupported statuses, invalid time ranges and overlapping active appointments.

WHY THIS MATTERS

A booking form can warn the user about a problem, but two people could still attempt to save the same time at almost the same moment.

The database is the final source of truth.

Database constraints make sure invalid or overlapping active appointments cannot be saved, even when requests arrive close together.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The appointments table exists.
- ☒ Every required column exists.
- ☒ Every optional column exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

The public.appointments table already exists.

Create one complete SQL file named:

appointment-constraints.sql

Return the complete file only.

Do not return snippets or partial SQL.

The SQL must:

- reject blank client_name values
- reject blank service values
- allow only booked, confirmed, completed or cancelled as status values
- require ends_at to be later than starts_at
- create the btree_gist extension if it is not already available
- add a database exclusion constraint that stops overlapping appointment time ranges for the same user
- allow a cancelled appointment to stop blocking that time

Use tstzrange(starts_at, ends_at, '[]') so an appointment may begin exactly when another appointment ends.

Make the SQL safe to run after the table has been created.

Add beginner-friendly comments explaining what each rule protects.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return one complete file:

- appointment-constraints.sql

The file should protect the appointment data and prevent overlapping active bookings.

SAVE YOUR FILES

Open Notepad.

Paste the complete SQL returned by AI.

Save the file inside your project folder as:

appointment-constraints.sql

Choose All Files and confirm that the filename does not end with .txt.

TEST YOUR WORK

Run appointment-constraints.sql in the Supabase SQL Editor.

Confirm that:

- ☒ Blank client names and services are rejected.
- ☒ Only booked, confirmed, completed and cancelled are accepted.
- ☒ The end must be later than the start.
- ☒ Overlapping active appointments for the same user are rejected.
- ☒ Back-to-back appointments are allowed.
- ☒ Cancelled appointments do not block a time.
- ☒ Row Level Security remains enabled.

CHECKPOINT

Before moving on, confirm that:

- ☒ The constraints were created successfully.
- ☒ Valid appointments are accepted.
- ☒ Invalid and overlapping appointments are rejected.
- ☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

A common mistake is checking for overlaps only in JavaScript.

That check is helpful for the user, but it cannot protect two requests that reach the database at almost the same time.

Another mistake is treating an appointment that ends at 10:00 and another that begins at 10:00 as overlapping.

The `[]` time range allows that useful back-to-back schedule.

BEHIND THE SCENES

Your application will later display filters based on status.

If every record stores different values, those filters become unreliable.

Using consistent values from the beginning prevents unnecessary problems later.

THINK LIKE A SOFTWARE DESIGNER

Good software should encourage consistent information.

Small decisions made early in a project often make future development much simpler.

WHAT YOU LEARNED

In this lesson you learned how to:

- improve data quality
- use consistent values
- prepare the database for reliable searching and filtering

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 4

ENABLING ROW LEVEL SECURITY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will confirm that Row Level Security is enabled for the appointments table.

This is one of the most important security features in the entire application.

WHY THIS MATTERS

Every authenticated user will store appointments inside the same database table.

Without Row Level Security, users could potentially access appointments that belong to someone else.

Row Level Security prevents this.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The appointments table exists.
- ☒ Authentication is working.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open your Supabase project.

Open:

Table Editor

Select:
appointments
Confirm that:
Row Level Security
is enabled.
If it is already enabled, do not change anything.
Do not disable it.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Confirm that:
☒ Row Level Security is enabled.

CHECKPOINT

Before moving on, confirm that:
☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

Some beginners disable Row Level Security while trying to solve another problem.
Do not do this.
Throughout this workbook, Row Level Security should remain enabled.

BEHIND THE SCENES

Authentication identifies the current user.

Row Level Security decides what that user is allowed to access.

Both are required for secure software.

THINK LIKE A SOFTWARE DESIGNER

Security should be built into the application from the beginning.

It should never become an afterthought.

WHAT YOU LEARNED

In this lesson you learned how to:

- confirm Row Level Security
- understand its relationship with authentication
- prepare the database for secure appointments

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 5

CREATING THE SELECT POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the SELECT policy for the appointments table.

This policy controls which appointments a user is allowed to view.

WHY THIS MATTERS

Without a SELECT policy, users cannot safely retrieve appointments.

The policy ensures that every user sees only their own appointments.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Row Level Security is enabled.
- ☒ The appointments table exists.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open:

Authentication

Then:

Policies

Locate the:

appointments

table.

Create a new policy.

Choose:

SELECT

Policy Name

Users can view only their own appointments

Use this policy expression exactly:

`user_id = auth.uid()`

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Return to the Policies page.

Confirm that:

☒ The appointments table now contains one SELECT policy.

Verify that:

☒ The policy applies to SELECT.

☒ The policy expression is:

```
user_id = auth.uid()
```

CHECKPOINT

Before moving on, confirm that:

- ✓ The SELECT policy exists.
- ✓ The policy has been saved.
- ✓ The policy compares:

```
user_id
```

```
with
```

```
auth.uid()
```

COMMON BEGINNER MISTAKES

A common mistake is comparing the wrong field.

Always compare:

```
user_id
```

```
with:
```

```
auth.uid()
```

No other comparison should be used.

BEHIND THE SCENES

Whenever your application requests appointments, Supabase checks this policy before returning any data.

Only rows belonging to the authenticated user will be returned.

THINK LIKE A SOFTWARE DESIGNER

Security should happen automatically.

Your application should not have to remember to protect every request.

The database should enforce those rules itself.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a SELECT policy
- protect appointments
- allow users to view only their own information

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 6

CREATING THE INSERT POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the INSERT policy.

This policy controls which new appointments may be saved.

WHY THIS MATTERS

Every new appointment should belong to the authenticated user who created it.

The INSERT policy ensures that only authorised records can be stored.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Open:

Authentication

Then:

Policies

Locate:

appointments

Create a new policy.

Choose:

INSERT

Policy Name

Users can insert only their own appointments

Use this policy expression exactly:

`user_id = auth.uid()`

Review the policy.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Return to the Policies page.

Confirm that your appointments table now contains:

☒ SELECT

☒ INSERT

Open the INSERT policy.

Verify:

☒ The policy applies to INSERT.

☒ The expression is:

```
user_id = auth.uid()
```

CHECKPOINT

Before moving on, confirm that:

☒ The INSERT policy exists.

☒ The policy has been saved successfully.

COMMON BEGINNER MISTAKES

Some beginners believe that authentication alone allows records to be inserted.

Authentication identifies the user.

The INSERT policy gives permission to save the record.

Both are required.

BEHIND THE SCENES

Whenever your application creates a new appointment, Supabase checks this policy before saving the record.

If the appointment belongs to the authenticated user, the record is accepted.

Otherwise, it is rejected.

THINK LIKE A SOFTWARE DESIGNER

Every database action should have its own security rule.

Viewing data and saving data are two different permissions.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an INSERT policy

- protect new appointments
- understand how authentication and Row Level Security work together

In the next lessons, you will create the UPDATE policy and secure cancellation function before carrying out a complete security audit.

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 7

CREATING THE UPDATE POLICY

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create the UPDATE policy for the appointments table.

This policy controls which appointments users are allowed to modify.

Once this lesson is complete, authenticated users will be able to update only the appointments they own.

WHY THIS MATTERS

Appointment information changes over time.

An appointment may:

- Change their phone number.
- Change their email address.
- Move to another date.
- Request a different service.
- Receive additional notes.

Your application should allow those changes while ensuring users cannot modify another person's appointments.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Row Level Security is enabled.
- ☒ The SELECT policy exists.

- ☒ The INSERT policy exists.
- ☒ The appointments table exists.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Instead, create the UPDATE policy directly inside Supabase.

Open your Supabase project.

From the left-hand menu, click:

- Authentication

Then click:

- Policies

Locate the:

- appointments

table.

Create a new policy.

Choose:

UPDATE

Policy Name

Users can update only their own appointments

Use the following policy expression exactly:

```
user_id = auth.uid()
```

Review the policy carefully.

Click:

Save Policy

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

This lesson is completed inside Supabase.

SAVE YOUR FILES

No project files are created or updated during this lesson.

TEST YOUR WORK

Return to the Policies page.

Confirm that your appointments table now contains:

☒ SELECT

☒ INSERT

☒ UPDATE

Open the UPDATE policy.

Verify that:

☒ The policy applies to UPDATE.

☒ The policy name is correct.

☒ The policy expression is:

`user_id = auth.uid()`

CHECKPOINT

Before moving on, confirm that:

☒ The UPDATE policy exists.

☒ The UPDATE policy has been saved.

☒ The policy compares:

```
user_id
```

```
with
```

```
auth.uid()
```

COMMON BEGINNER MISTAKES

Some beginners believe that identifying an appointment by its ID is enough when updating it.

It is not.

Later in this workbook, your JavaScript will update appointments by checking both:

- The appointment ID
- The authenticated user's ID

The database will then apply the UPDATE policy as an additional layer of protection.

BEHIND THE SCENES

Every time someone edits an appointment, Supabase checks the UPDATE policy before saving the changes.

If the record belongs to the authenticated user, the update succeeds.

If it belongs to another user, the request is rejected automatically.

THINK LIKE A SOFTWARE DESIGNER

Reading information and changing information are different permissions.

Professional software treats them separately.

Just because someone can see an appointment does not automatically mean they should be allowed to change it.

WHAT YOU LEARNED

In this lesson you learned how to:

- create an UPDATE policy
- protect appointments during editing
- understand why updates require their own permission

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 8

CREATING SECURE APPOINTMENT CANCELLATION

Estimated Time

20 minutes

WHAT YOU ARE BUILDING

In this lesson, you will create one secure database function that cancels an appointment.

Cancellation changes the status to cancelled and keeps the record in the schedule.

The function will work only for the authenticated owner.

WHY THIS MATTERS

Businesses often need a history of cancelled appointments.

Permanently removing the row would erase useful information and make statistics less accurate.

Soft cancellation preserves the history while releasing the time for a new booking.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The SELECT policy exists.
- ☒ The INSERT policy exists.
- ☒ The UPDATE policy exists.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

The appointments table already has Row Level Security and user-owned SELECT, INSERT and UPDATE policies.

Create one complete SQL file named:

`cancel-appointment-function.sql`

Return the complete file only.

Do not return snippets or partial SQL.

Create a `public.cancel_appointment` function that:

- accepts one appointment ID
- uses the authenticated user's `auth.uid()`
- updates only an appointment owned by that user
- changes status to cancelled
- updates `updated_at`
- does not permanently delete the appointment
- returns the updated appointment
- reports a clear error when the appointment does not exist, belongs to another user or is already cancelled

Use security invoker so the normal Row Level Security rules still apply.

Set a safe `search_path`.

Grant execution to authenticated users only.

Add simple comments explaining each security decision.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return one complete file:

- `cancel-appointment-function.sql`

The file should create secure soft cancellation without adding a secure cancellation function.

SAVE YOUR FILES

Open Notepad.

Paste the complete SQL returned by AI.

Save the file inside your project folder as:

cancel-appointment-function.sql

Choose All Files and confirm that the filename ends with .sql.

TEST YOUR WORK

Run cancel-appointment-function.sql in the Supabase SQL Editor.

Confirm that:

- ☒ cancel_appointment exists.
- ☒ It uses security invoker and checks auth.uid().
- ☒ It updates status and updated_at.
- ☒ It does not permanently remove a row.
- ☒ Only authenticated users may execute it.

CHECKPOINT

Before moving on, confirm that:

- ☒ The secure cancellation function exists.
- ☒ Row Level Security remains enabled.
- ☒ No appointment deletion policy was added.
- ☒ Cancellation preserves the appointment record.

COMMON BEGINNER MISTAKES

A common mistake is permanently removing a cancelled booking.

That loses useful business history.

Another mistake is using a function that bypasses Row Level Security.

This function uses security invoker so the existing security rules continue protecting every update.

BEHIND THE SCENES

The function performs a normal database update.

It finds one appointment whose ID and user ID match the request, changes the status and returns the updated record.

If ownership does not match, no appointment is changed.

THINK LIKE A SOFTWARE DESIGNER

Deleting information should always receive special attention.

Professional software protects permanent actions with multiple layers of security instead of assuming every request is valid.

WHAT YOU LEARNED

In this lesson you learned how to:

- create a secure cancellation function
- protect appointments from unauthorised cancellation
- complete the ownership policies and secure cancellation support

CHAPTER 4

BUILDING THE APPOINTMENT DATABASE

LESSON 9

TESTING THE APPOINTMENT DATABASE

Estimated Time

30 minutes

WHAT YOU ARE BUILDING

In this lesson, you will verify that your appointment database has been built correctly.

You are not creating new tables or policies.

Instead, you will review everything completed so far before appointment management begins.

WHY THIS MATTERS

The appointment database is the foundation of every feature that follows.

If there is a mistake in the database structure or security policies, adding, editing and cancelling appointments may not work correctly.

Professional developers review their database carefully before writing application features.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The appointments table exists.
- ☒ SELECT, INSERT and UPDATE policies exist.
- ☒ The cancellation function exists.
- ☒ The time and status constraints exist.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with AI in this lesson.

Review the appointments table, its constraints, the SELECT, INSERT and UPDATE policies, and the cancel_appointment function.

Confirm that every ownership rule compares user_id with auth.uid().

Do not create a permanent-deletion policy because this workbook keeps cancelled appointments as business history.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No project files are created or updated.

TEST YOUR WORK

Complete the database tests with two different user accounts.

Use the application or the authenticated browser session. Do not place another user's ID into a public test page.

Confirm that:

- ☒ Account A can create and read its own appointments.
- ☒ Account B can create and read its own appointments.
- ☒ Account A cannot read or update Account B's appointments.
- ☒ Blank names and services are rejected.
- ☒ An end time before the start time is rejected.
- ☒ Unsupported status values are rejected.
- ☒ Overlapping active appointments for one user are rejected.

- ✓ An appointment may begin exactly when the previous one ends.
 - ✓ Cancelling an appointment keeps the record but changes its status.
 - ✓ The cancelled time can be booked again.
 - ✓ Row Level Security remains enabled.
- Remove any temporary test appointments when the testing is complete.

CHECKPOINT

Before moving on, confirm that:

- ✓ Every required column exists.
- ✓ Valid statuses are protected.
- ✓ Invalid and overlapping times are rejected.
- ✓ SELECT, INSERT and UPDATE ownership policies work.
- ✓ Secure cancellation works.
- ✓ Two-account privacy is protected.

COMMON BEGINNER MISTAKES

A common mistake is testing with only one account.

That proves the application can save information, but it does not prove privacy.

Always test with two accounts and confirm that neither account can read or change the other account's appointments.

BEHIND THE SCENES

Your appointment database is now fully prepared.

The next chapter will not change the database.

Instead, it will begin using this database to store and retrieve real appointment information.

Because the database already protects every row, your application can focus on providing a good user experience while Supabase continues enforcing security behind the scenes.

THINK LIKE A SOFTWARE DESIGNER

A secure application begins with a secure database.

When the foundation is built correctly, every feature added later becomes easier to develop and easier to trust.

Professional developers invest time in getting the database right before writing business logic.

WHAT YOU LEARNED

In this lesson you learned how to:

- review an appointment database
- verify Row Level Security
- verify every database policy
- confirm that the application is ready for appointment management

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System now has a secure database capable of storing complete appointment details.

You designed a clear data model and created fields suitable for a real appointment schedule.

You also enabled Row Level Security, created SELECT, INSERT and UPDATE policies, and added secure soft cancellation.

Every authenticated user will be able to:

- ✓ View only their own appointments.
- ✓ Create only their own appointments.
- ✓ Update only their own appointments.
- ✓ Cancel only their own appointments.

The database will automatically reject requests that do not satisfy these security rules.

CHAPTER MILESTONE

By the end of Chapter 4, you have successfully built:

- ✓ appointments table
- ✓ Rich appointment data model
- ✓ Required and optional fields
- ✓ Primary Key
- ✓ User ownership
- ✓ Row Level Security
- ✓ SELECT policy
- ✓ INSERT policy
- ✓ UPDATE policy
- ✓ secure cancellation function
- ✓ Secure database foundation

TRANSITION TO CHAPTER 5

Your Appointment Booking System now has everything it needs behind the scenes.

Users can create accounts, sign in securely, and the database is ready to store appointment information.

The next step is to build the appointment dashboard.

You will create a professional appointment entry form, introduce richer business forms, detect overlapping appointments before saving, and begin building business-oriented dashboard statistics.

This chapter introduces useful software design patterns for appointment booking.

CHAPTER 5

BUILDING THE APPOINTMENT DASHBOARD

CHAPTER INTRODUCTION

The secure database is ready.

You can now build the private dashboard where a signed-in user books appointments and sees a quick summary of the schedule.

The booking form will collect client details, service, start and end times, status and notes.

The browser will provide a helpful preview, but Supabase will remain responsible for ownership and overlap protection.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Protected appointment dashboard
- Appointment booking form
- Client contact fields
- Service, date and time fields
- Status and notes
- Date and time validation
- Secure appointment saving
- Friendly overlap messages
- Total, today's, upcoming and cancelled statistics

LESSON 1

DESIGNING THE APPOINTMENT DASHBOARD

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will design the main working area of your Appointment Booking System.

This dashboard is where users will spend most of their time after signing in.

Rather than displaying a blank page, the dashboard will present useful business information alongside an organised appointment management area.

The appointment form and business statistics you build later in this chapter will fit naturally into this layout.

WHY THIS MATTERS

Business software is used repeatedly throughout the working day.

A good dashboard helps users understand where they are, what they can do next and how the business is performing.

A poorly organised dashboard slows people down.

Before adding appointment management features, it is worth investing time in creating a clean, professional workspace.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Registration works correctly.
- ☒ Email verification works.
- ☒ Login works.
- ☒ Dashboard authentication protection works.

✓ Logout works.

Only authenticated users should be able to reach the dashboard.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Appointment Booking System.

Everything built so far must continue working exactly as before.

Continue using the existing project architecture, including the shared `supabaseClient` from `supabase-config.js`.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `dashboard.html`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Transform the protected dashboard into a clean, professional appointment management workspace.

Do not build appointment saving yet.

Do not connect the appointment form to Supabase yet.

This lesson focuses only on the dashboard layout and user experience.

DASHBOARD.HTML

Keep the existing authentication behaviour.

Retain:

- User email display
- Logout button

Create a professional dashboard containing the following sections.

Dashboard Header

Display:

- Page title:

Appointment Dashboard

- Welcome message for the authenticated user
- Logout button

Business Statistics Section

Create four professional summary cards.

Each card should contain:

- Large number
- Descriptive label

Prepare cards for:

- Total Appointments
- Today's Appointments
- Upcoming Appointments
- Cancelled Appointments

Use placeholder values of:

0

Give every statistic a unique ID so JavaScript can update them later.

Appointment Management Section

Create a clearly separated section titled:

Add New Appointment

Under this heading, prepare space for a professional appointment form.

Do not connect the form to JavaScript yet.

Include fields for:

- Appointment Status
- Client Name
- Service
- Email

- Phone
- Service
- Appointment date and time
- Appointment Date
- Start Time
- End Time
- Status
- Notes

Include:

Book Appointment

button.

Give every field a clear ID.

Appointment Schedule Section

Below the appointment form, create another section titled:

Appointment Schedule

Display a simple placeholder message:

Your appointments will appear here after you begin adding appointments.

Give this section a clear container ID for future lessons.

Status Message Area

Create one status message area above the appointment form.

This area will later display:

- Success messages
- Validation messages
- Loading messages

Hide it initially.

DASHBOARD.JS

Keep:

- Authentication protection
- Logout

- Authenticated user's email display

Add safe references for:

- Appointment form
- Status message
- Dashboard statistics
- Appointment Schedule

Do not:

- Save appointments
- Validate fields
- Connect to Supabase
- Display appointment cards

The page should load without JavaScript errors.

AUTH.CSS

Keep every existing style.

Design a modern business dashboard.

Style:

- Dashboard header
- Summary cards
- Appointment form container
- Appointment Schedule container
- Section headings
- Form labels
- Input fields
- Text area
- Book Appointment button
- Status message area

The design should feel clean and spacious.

Use generous spacing.

Keep the layout fully responsive.

On smaller screens:

- Stack dashboard cards neatly.

- Keep the appointment form comfortable to complete.
 - Prevent horizontal scrolling.
- Do not introduce unnecessary visual effects.

IMPORTANT

- Everything already built must continue working.
- Do not build appointment-saving logic.
- Do not build validation.
- Do not build dashboard statistics.
- Do not retrieve appointments.
- Those features belong to later lessons.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now provide a professional workspace ready for appointment management.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

- Before replacing an existing file, copy your complete project folder.
- Save the copy outside your working project folder.
- Name the backup clearly using the current chapter and lesson.
- Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

in Notepad.

Replace its complete contents.

Save the file.

Repeat the same process for:

dashboard.js

and

auth.css.

TEST YOUR WORK

Sign in.

Open the dashboard.

Confirm that:

- ☒ The dashboard header appears.
- ☒ The welcome message displays correctly.
- ☒ Four business statistic cards appear.
- ☒ The Add New Appointment section appears.
- ☒ Every appointment field appears.
- ☒ The Book Appointment button appears.
- ☒ The Appointment Schedule placeholder appears.
- ☒ Logout still works.

Resize the browser window.

Confirm that:

- ☒ Dashboard cards remain organised.
- ☒ The appointment form remains readable.

- ✓ No unnecessary horizontal scrolling appears.

CHECKPOINT

Before moving on, confirm that:

- ✓ The dashboard layout is complete.
- ✓ All appointment fields are visible.
- ✓ The dashboard remains protected.
- ✓ Existing authentication continues working.
- ✓ Every future section has a clear placeholder.

COMMON BEGINNER MISTAKES

Some beginners immediately begin connecting forms to the database before finishing the dashboard layout.

Doing so makes it much harder to recognise whether a problem comes from the design or the JavaScript.

Complete the structure first.

Then add behaviour.

BEHIND THE SCENES

Professional software is usually developed in layers.

The first layer provides the visual structure.

The next layer adds behaviour.

The final layer connects the interface to the database.

Separating those responsibilities makes the software easier to build and maintain.

THINK LIKE A SOFTWARE DESIGNER

A good dashboard should answer three questions immediately:

Where am I?

What can I do here?

What information is most important?

Design every dashboard with those questions in mind.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports designing the appointment dashboard. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional business dashboard
- organise information into logical sections
- prepare a dashboard for future functionality
- design software before connecting it to a database

CHAPTER 5

BUILDING THE APPOINTMENT DASHBOARD

LESSON 2

BUILDING THE APPOINTMENT BOOKING FORM

Estimated Time

70 minutes

WHAT YOU ARE BUILDING

In the previous lesson, you designed the Appointment Dashboard and prepared space for the appointment form.

In this lesson, you will build a professional appointment entry form that businesses can use to collect appointment information consistently.

The form will validate user input, clean appointment information automatically, and prepare every appointment for secure storage.

The information will not be saved into Supabase yet.

Saving appointments will be completed in the next lesson.

WHY THIS MATTERS

Good appointments begin with good data entry.

If users can enter incomplete or inconsistent information, the appointment database quickly becomes difficult to search and maintain.

A professional appointment form helps prevent those problems before information reaches the database.

BEFORE YOU CONTINUE

Confirm that:

- ☒ The Appointment Dashboard appears correctly.

- ✓ All dashboard sections are visible.
- ✓ The Appointment Schedule placeholder appears.
- ✓ Dashboard authentication still works.
- ✓ Logout still works.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Appointment Booking System with authentication, a protected dashboard and a shared supabaseClient from supabase-config.js.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build a complete beginner-friendly appointment booking form.

DASHBOARD.HTML

Keep the protected dashboard and logout feature.

Add clearly labelled fields for:

- Client Name – required
- Client Email – optional, type email
- Client Phone – optional, type tel
- Service – required
- Appointment Date – required, type date
- Start Time – required, type time

- End Time – required, type time
 - Status – required, with Booked and Confirmed options
 - Notes – optional textarea
- Add:
- Book Appointment button
 - Form message area with `aria-live="polite"`
 - A simple appointment summary that updates while the user types

DASHBOARD.JS

- Keep authentication and logout working.
- Add browser-side validation.
- Require client name, service, date, start time and end time.
- Confirm the end is later than the start.
- Do not save yet.
- Create complete ISO date-time values from the selected local date and times using the browser Date object and `toISOString()`.
- Keep the ISO values ready for the saving lesson.
- Never ask the learner to edit generated code manually.

AUTH.CSS

- Keep all existing styles.
- Style the complete form, validation messages and live summary.
- Make the form easy to use on a small screen.

IMPORTANT

- Return complete updated files.
- Do not include explanations inside the file contents.
- Do not use frameworks or build tools.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The appointment form should now validate and prepare appointment information professionally.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Open:

dashboard.html

in Notepad.

Replace its complete contents.

Save the file.

Repeat the same process for:

dashboard.js

and

auth.css.

TEST YOUR WORK

Open the protected dashboard.

Enter a client name, service, date, start time and end time.

Confirm that:

- ☒ Required fields are enforced.
- ☒ Optional email, phone and notes may be empty.
- ☒ Email validation works when an email is entered.
- ☒ The end must be later than the start.
- ☒ The live summary is clear.
- ☒ No appointment is saved during this lesson.
- ☒ The form remains usable on a small screen.

CHECKPOINT

Before moving on, confirm that:

- ☒ Validation works.
- ☒ Required fields work.
- ☒ Email validation works.
- ☒ Phone validation works.
- ☒ Appointment information is prepared.
- ☒ No appointment has been saved yet.

COMMON BEGINNER MISTAKES

Many beginners validate appointment information after saving it.

Professional software validates and cleans information before it ever reaches the database.

Doing so produces more consistent appointments and reduces future problems.

BEHIND THE SCENES

Your appointment form now behaves like forms used in many professional business applications.

Instead of trusting users to enter perfectly formatted information, the software improves the quality of the data automatically before it can be stored.

THINK LIKE A SOFTWARE DESIGNER

Every appointment added today may still exist in the business database years from now.

Investing in clean data entry now saves countless hours of correction later.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building the appointment booking form. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional appointment form
- validate appointment information
- normalise appointment data
- prepare information for secure storage

CHAPTER 5

BUILDING THE APPOINTMENT DASHBOARD

LESSON 3

SAVING APPOINTMENTS SECURELY

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

Your appointment form can now collect high-quality information.

In this lesson, you will complete the entire appointment creation workflow.

The application will check for overlapping appointments, save authorised appointment information into Supabase, reset the form, and immediately refresh the business dashboard.

By the end of this lesson, your Appointment Booking System will be capable of securely creating new appointments just like a real business application.

WHY THIS MATTERS

Creating appointments is one of the most important responsibilities of this software.

The workflow must protect data quality, maintain appointment privacy, and ensure that every record belongs only to the authenticated user.

Once this lesson is complete, businesses will be able to trust the appointment information stored inside the application.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Appointment form validation works.
- ☒ Appointment information is prepared.
- ☒ The appointments table exists.
- ☒ Row Level Security remains enabled.

- ✓ INSERT policy exists.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

The Appointment Booking System has a working booking form and a secure appointments table.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.js

GENERAL GOAL

Save one complete appointment securely.

Keep all existing validation and authentication.

When the form is submitted:

- get the authenticated user
- stop if no user is signed in
- trim text fields
- convert the chosen local date and times to ISO strings
- confirm ends_at is later than starts_at
- insert into appointments

Insert:

- user_id from the authenticated user
- client_name
- client_email or null
- client_phone or null
- service
- starts_at

- `ends_at`

- `status`

- `notes` or `null`

Never trust a user ID from the page address or form.

Continue relying on Row Level Security and database constraints.

Disable the submit button while saving.

If Supabase reports an overlapping appointment, show a friendly message explaining that the selected time is unavailable.

Do not display raw database errors.

Use `console.error()` for technical details.

After success:

- show Appointment booked successfully.

- reset the form

- refresh the schedule

- refresh dashboard statistics

IMPORTANT

Return the complete updated `dashboard.js` file only.

Do not return a snippet.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

AI should return one complete updated file:

- `dashboard.js`

The file should save appointments securely and handle overlap errors clearly.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in and book a valid appointment.

Confirm that:

- ☒ The appointment saves with the authenticated user's ID.
- ☒ starts_at and ends_at contain complete ISO values.
- ☒ The form resets and repeated submissions are prevented.

Try to book an overlapping active appointment.

Confirm that:

- ☒ Supabase rejects it.
- ☒ The page shows a friendly unavailable-time message.
- ☒ No partial or repeated appointment is saved.

CHECKPOINT

Before moving on, confirm that:

- ☒ Valid appointments save securely.
- ☒ Overlapping active appointments are rejected.
- ☒ Optional contact fields work.
- ☒ Ownership comes from the authenticated user.
- ☒ Row Level Security remains enabled.

COMMON BEGINNER MISTAKES

A common mistake is checking duplicate records across the entire database.

Appointment creation should compare only records belonging to the authenticated user.

Another common mistake is trusting the browser to supply the user ID.

Always obtain the authenticated user's ID directly from Supabase before creating the appointment.

BEHIND THE SCENES

The appointment creation workflow now follows the same sequence used in many professional business systems.

Information is validated, cleaned, checked for duplicates, stored securely, and immediately reflected throughout the application.

Each step builds on the previous one to maintain reliable business data.

THINK LIKE A SOFTWARE DESIGNER

Creating an appointment is more than inserting information into a database.

It is a carefully controlled business process that protects data quality, user privacy and future reporting.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports saving appointments securely. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- detect overlapping appointments
- save appointments securely
- protect appointment ownership
- refresh application data automatically
- complete a professional appointment creation workflow

CHAPTER 5

BUILDING THE APPOINTMENT DASHBOARD

LESSON 4

BUILDING APPOINTMENT DASHBOARD STATISTICS

Estimated Time

60 minutes

WHAT YOU ARE BUILDING

Your Appointment Booking System can now create appointments securely.

In this lesson, you will transform the dashboard from a simple data entry screen into a useful business dashboard.

The dashboard will automatically calculate and display meaningful appointment statistics every time the page loads and whenever a new appointment is added.

Instead of forcing users to count appointments themselves, the software will do the work automatically.

WHY THIS MATTERS

A useful dashboard answers important questions immediately.

The user should see the size of the schedule, today's workload, future appointments and cancellation activity without counting records manually.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Appointments save successfully.
- ☒ Overlap protection works.
- ☒ The appointment form clears after saving.
- ☒ The appointments table contains several records.
- ☒ Dashboard authentication still works.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

Appointments now save securely.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build live appointment dashboard statistics.

Display four cards:

- Total Appointments
- Today's Appointments
- Upcoming Appointments
- Cancelled Appointments

Use the authenticated user's appointments only.

Use the browser's local date when deciding whether an appointment is today.

Count upcoming appointments whose starts_at is later than now and whose status is not cancelled.

Show loading, empty and error states.

Refresh the statistics after booking, editing or cancelling an appointment.

Do not expose another user's data.

Continue relying on Row Level Security.

Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should automatically display live appointment statistics.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Create appointments for today and future dates, then cancel one appointment.

Confirm that:

- ☒ Total Appointments is correct.
- ☒ Today's Appointments uses the local calendar date.
- ☒ Upcoming Appointments excludes cancelled records.
- ☒ Cancelled Appointments is correct.

- ✓ Statistics refresh after changes.
- ✓ Only the authenticated user's appointments are counted.

CHECKPOINT

Before moving on, confirm that:

- ✓ All four statistics are accurate.
- ✓ Local-date comparisons work.
- ✓ Statistics refresh automatically.
- ✓ Appointment saving still works.
- ✓ Account privacy remains protected.

COMMON BEGINNER MISTAKES

- Some beginners manually increase the displayed numbers after saving an appointment.
- Professional software recalculates statistics from the database.
- This guarantees that the dashboard always reflects the actual appointments stored in the system.

BEHIND THE SCENES

- The dashboard does not store separate statistics.
- Instead, it calculates them from the appointments already stored in the database.
- Whenever new information is added, the calculations simply run again using the latest data.
- This keeps the dashboard accurate without storing duplicate information.

THINK LIKE A SOFTWARE DESIGNER

- A dashboard should help users make decisions quickly.
- Whenever information can be calculated automatically from existing data, let the software do the work instead of expecting users to calculate it themselves.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building appointment dashboard statistics. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- calculate business statistics
- display live dashboard information
- refresh statistics automatically
- improve business visibility through software

CHAPTER SUMMARY

Congratulations.

Your protected dashboard can now collect, validate and save complete appointments.

Supabase protects ownership and rejects overlapping active times.

The dashboard also calculates total, today's, upcoming and cancelled statistics from the authenticated user's saved appointments.

CHAPTER MILESTONE

By the end of Chapter 5, you have successfully built:

- ✓ Protected appointment dashboard
- ✓ Complete booking form
- ✓ Local date and time preparation
- ✓ Secure appointment saving
- ✓ Overlap error handling
- ✓ Automatic form reset
- ✓ Live appointment statistics
- ✓ Automatic dashboard refresh

TRANSITION TO CHAPTER 6

Your application can now book appointments securely.

In the next chapter, you will turn those saved records into a clear date-grouped schedule and a protected Appointment Details page.

CHAPTER 6

VIEWING APPOINTMENTS

CHAPTER INTRODUCTION

The application can now save appointments and calculate useful statistics.

The next step is to help the user see the schedule clearly.

You will display appointments in time order, group them by date and provide a separate details page for one authorised appointment.

Clear loading, empty and error states will help beginners understand what the application is doing.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Appointment Schedule
- Date-grouped appointment cards
- Local date and time display
- Today, upcoming and past labels
- Clear status badges
- Appointment Details page
- Call and email actions when contact details exist
- Loading, empty and error states
- Responsive schedule testing
- Two-account privacy testing

LESSON 1

BUILDING THE APPOINTMENT SCHEDULE

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete Appointment Schedule.

The application will retrieve appointments from Supabase, display them as professional appointment cards, generate client initials automatically, provide click-to-call and click-to-email actions, and allow users to open individual appointment details.

The schedule will also provide friendly loading, empty and error states to improve the overall user experience.

WHY THIS MATTERS

Businesses rarely interact directly with database tables.

Instead, they work with organised appointment lists that present information clearly and make common tasks quick and convenient.

A well-designed appointment schedule allows users to recognise appointments immediately and perform common actions with very little effort.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Appointments save successfully.
- ☒ Dashboard statistics work.
- ☒ Several appointments exist.
- ☒ The Appointment Schedule placeholder appears.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Appointment Booking System with authentication, secure appointment saving and dashboard statistics.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build the Appointment Schedule on the protected dashboard.

Load the authenticated user's appointments from Supabase.

Select only the fields needed for the schedule.

Order by starts_at from earliest to latest.

Store the returned appointments in a reusable array.

Group appointments under clear date headings.

Format dates and times for the user with Intl.DateTimeFormat().

Never display raw ISO values.

Each appointment card should show:

- client name
- service
- start and end time
- status badge
- View Appointment button

The button should open:

appointment-details.html?id=APPOINTMENT_ID

Add separate loading, empty, error and content states.

Cancelled appointments should remain visible with a clear cancelled style.

Refresh the schedule after a new appointment is saved.

Continue relying on Row Level Security.

Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The dashboard should now display a complete Appointment Schedule.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Sign in.

Confirm that:

- ✓ Appointment cards appear.
- ✓ Initials display correctly.
- ✓ Phone numbers are clickable.
- ✓ Email addresses are clickable.
- ✓ View Appointment buttons contain appointment IDs.
- ✓ Loading state works.
- ✓ Empty state works.
- ✓ Appointment cards refresh after adding another appointment.
- ✓ No duplicate cards appear.
- ✓ Only authenticated user's appointments appear.

CHECKPOINT

Before moving on, confirm that:

- ✓ Appointment Schedule works correctly.
- ✓ Appointment cards display correctly.
- ✓ Contact actions work.
- ✓ Loading, empty and error states work.
- ✓ Appointment privacy remains protected.

COMMON BEGINNER MISTAKES

- Some beginners retrieve every appointment from the database and then filter them inside JavaScript.
- Always request only the authenticated user's records.
- Doing so improves both security and performance.

BEHIND THE SCENES

- Each appointment card is created from one appointment returned by Supabase.

JavaScript converts database information into a visual component that users can recognise and interact with immediately.

THINK LIKE A SOFTWARE DESIGNER

Business software should present information in a way that reduces effort.

A well-designed appointment schedule allows users to recognise appointments quickly and perform common actions without unnecessary navigation.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building the appointment schedule. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a professional Appointment Schedule
- display appointment cards
- generate client initials
- add contact actions
- handle loading, empty and error states
- refresh displayed information automatically

CHAPTER 6

VIEWING APPOINTMENTS

LESSON 2

BUILDING THE APPOINTMENT DETAILS PAGE

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

The Appointment Schedule now allows users to see every appointment at a glance.

In this lesson, you will build the complete Appointment Details page.

Instead of displaying only summary information, the profile page will securely retrieve one appointment from Supabase and display every available detail in a clean, professional layout.

Users will also be able to call appointments, send emails, and begin the appointment editing process from this page.

WHY THIS MATTERS

Appointment cards are useful for browsing information quickly, but businesses often need to see the complete appointment before making decisions.

A dedicated profile page provides enough space to display contact information, addresses, business information, notes and record details without overcrowding the Appointment Schedule.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Appointment Schedule works correctly.
- ☒ View Appointment buttons appear.
- ☒ Appointment cards display correctly.

- ✓ Appointment IDs are passed through the page address.
- ✓ appointment-details.html already exists.
- ✓ appointment-details.js already exists.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

- The protected dashboard now displays an Appointment Schedule.
- Everything already built must continue working.
- Return complete files only.
- Do not return snippets or partial code.
- Create or update:
 - appointment-details.html
 - appointment-details.js
 - auth.css

GENERAL GOAL

- Build a protected page that displays one complete authorised appointment.
- Read only the id value from the page address.
- Get the authenticated user.
- Retrieve one appointment where:
 - id matches the page address
 - user_id matches the authenticated user
- Continue relying on Row Level Security.
- Display:
 - client name
 - client email when available
 - client phone when available
 - service
 - local appointment date

- local start and end time
- status
- notes when available
- created and updated times

Add:

- Edit Appointment
- Cancel Appointment
- Back to Dashboard
- Logout

Add call and email links only when those values exist.

Add loading, not-found, error and content states.

Never expose raw database errors.

Do not allow a signed-in user to view another user's appointment.

Return complete files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- appointment-details.html
- appointment-details.js
- auth.css

The application should now support complete appointment details.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open the Appointment Schedule.

Select:

View Appointment

Confirm that:

☒ Correct appointment details appears.

☒ Initials display correctly.

☒ Contact information displays correctly.

☒ Call Client works.

☒ Email Client works.

☒ Edit Appointment opens:

dashboard.html?edit=APPOINTMENT_ID

☒ Dates display correctly.

☒ Missing notes display the default message.

Attempt to access another user's appointment ID.

Confirm that:

☒ Appointment information is not displayed.

CHECKPOINT

Before moving on, confirm that:

☒ Appointment details loads correctly.

☒ Authentication works.

- ✓ Appointment ownership is protected.
- ✓ Contact actions work.
- ✓ Edit Appointment works.

COMMON BEGINNER MISTAKES

- Some beginners retrieve every appointment before selecting one inside JavaScript.
- Always request only the appointment that the page needs.
- Doing so improves security, performance and keeps the application easier to maintain.

BEHIND THE SCENES

- The appointment details page demonstrates one of the most common patterns in business software.
- Instead of creating one HTML page for every appointment, the application uses one reusable page that loads different information depending on the appointment ID supplied.

THINK LIKE A SOFTWARE DESIGNER

- Reusable pages make software easier to maintain.
- Whenever many records share the same layout, build one dynamic page instead of creating many separate pages.

CODE-READING QUESTION

Open appointment-details.js. Find one function that supports building the appointment details page. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build a complete appointment details
- retrieve one authorised appointment
- protect appointment privacy
- create reusable profile pages
- build professional appointment detail screens

CHAPTER 6

VIEWING APPOINTMENTS

LESSON 3

TESTING THE APPOINTMENT SCHEDULE

Estimated Time

45 minutes

WHAT YOU ARE BUILDING

In this lesson, you will thoroughly test the complete Appointment Schedule and Appointment Details system.

Rather than adding new features, you will confirm that every part of the schedule works correctly under different conditions.

Professional software development always includes careful testing before new features are added.

WHY THIS MATTERS

Many software problems appear only after several features begin working together.

Testing helps you discover those problems before users rely on the application.

This lesson will confirm that appointment privacy, schedule behaviour and profile pages all function correctly.

BEFORE YOU CONTINUE

Prepare:

- Two separate user accounts.
- Several Booked appointments.
- Several Confirmed appointments.

Include appointments with and without client email addresses.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

Correct any problems before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Sign in and create appointments on several dates.

Confirm that:

- ☒ Appointments appear in chronological order.
- ☒ Date headings are clear.
- ☒ Times display in the browser's local time.
- ☒ Cancelled appointments remain visible and clearly labelled.
- ☒ View Appointment opens the correct record.
- ☒ Missing email, phone and notes do not create broken areas.
- ☒ Loading, empty and error states work.
- ☒ The schedule remains usable on a small screen.

Complete a two-account privacy test.

Confirm that Account A cannot open Account B's appointment by changing the ID in the page address.

CHECKPOINT

Before completing this chapter, confirm that:

- ✓ Appointment Schedule works correctly.
- ✓ Appointment Detailss work correctly.
- ✓ Contact actions work.
- ✓ Authentication works.
- ✓ Appointment privacy works.
- ✓ Responsive layouts work.

COMMON BEGINNER MISTAKES

Many beginners assume that software works because one successful test passes.

Professional software is tested under many different situations, including missing information, different users and unusual screen sizes.

BEHIND THE SCENES

Appointment privacy depends on several layers working together:

- Authentication
- Secure queries
- Row Level Security
- Careful interface design

Removing any one of these layers weakens the application.

THINK LIKE A SOFTWARE DESIGNER

Good software is not only built carefully.

It is also tested carefully.

Professional developers spend significant time confirming that completed features continue working as new features are added.

WHAT YOU LEARNED

In this lesson you learned how to:

- test complete software features
- verify appointment privacy
- verify responsive layouts
- test authentication
- confirm professional software quality

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System now allows users to browse appointments through a professional Appointment Schedule and view complete appointment details without leaving the application.

The software retrieves only authorised appointments, protects appointment privacy, provides convenient contact actions and adapts smoothly to different screen sizes.

The application now resembles the type of appointment management software used by many businesses every day.

CHAPTER MILESTONE

By the end of Chapter 6, you have successfully built:

- ✓ Appointment Schedule
- ✓ Appointment Cards
- ✓ Appointment Initials
- ✓ Click-to-call
- ✓ Click-to-email
- ✓ Appointment Details Page
- ✓ Edit Appointment Action
- ✓ Loading State
- ✓ Empty State
- ✓ Error State
- ✓ Responsive Schedule
- ✓ Appointment Privacy Protection

TRANSITION TO CHAPTER 7

Your application can now create, organise and display appointment information professionally.

The next step is making those appointments much easier to find.

As appointment databases grow, scrolling through hundreds of appointment cards quickly becomes inefficient.

In the next chapter, you will build intelligent search, filtering and sorting capabilities that allow businesses to locate appointments within seconds.

CHAPTER 7

SEARCHING, FILTERING AND SORTING APPOINTMENTS

CHAPTER INTRODUCTION

Your Appointment Booking System can now create, store and display appointments professionally.

As the number of appointments grows, however, finding the right appointment becomes increasingly difficult.

Scrolling through hundreds of appointment cards is slow and frustrating.

Professional appointment management software solves this problem by giving users powerful tools for searching, filtering and organising information.

In this chapter, you will build a complete appointment discovery system that allows users to locate appointments within seconds.

The Appointment Schedule will become far more useful without changing the appointment information stored in the database.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Appointment Schedule toolbar
- Live appointment search
- Search by name
- Search by service

- Search by phone number
- Search by email address
- Appointment Status filtering
- Appointment sorting
- Result counter
- No-results state
- Clear Search and Filters
- Automatic schedule updates

LESSON 1

BUILDING APPOINTMENT SEARCH, FILTERING AND SORTING

Estimated Time

120 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build a complete Appointment Schedule toolbar.

Users will be able to:

- Search appointments while typing.
- Filter appointments by Appointment Status.
- Sort appointments.
- Clear all search controls instantly.

The Appointment Schedule will update automatically without reloading the page.

WHY THIS MATTERS

Businesses often manage hundreds or thousands of appointments.

Without search and filtering, finding the correct appointment quickly becomes difficult.

Professional software should help users locate information in seconds instead of forcing them to browse long lists manually.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Appointment Schedule works correctly.
- ☒ Appointment cards display correctly.
- ☒ Appointment details work correctly.

✓ Dashboard statistics work.

✓ Several appointments exist.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

The protected dashboard already displays a complete Appointment Schedule.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.html
- dashboard.js
- auth.css

GENERAL GOAL

Build a complete toolbar for finding appointments quickly.

Add:

- Search Appointments field
- Status filter with All, Booked, Confirmed, Completed and Cancelled
- Date filter with All Dates, Today, Upcoming and Past
- Sort with Soonest First, Latest First, Client A-Z and Client Z-A
- Clear Search and Filters button
- result information
- separate no-results state

Search while the user types across:

- client name
- service
- client email
- client phone

Ignore letter case and unnecessary spaces.

Filter and sort a copied array.

Do not request Supabase again for every key press.

Use the browser's local date for Today.

Cancelled appointments must be included only when the selected filters allow them.

Keep loading, empty, error and schedule states separate.

Continue supporting authentication, booking, statistics and appointment details.

Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- auth.css

The Appointment Schedule should now support professional searching, filtering and sorting.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Confirm that:

- ☒ Searching by client name works.
- ☒ Searching by service works.
- ☒ Searching by service works.
- ☒ Searching by phone works.
- ☒ Searching by email works.
- ☒ Appointment Status filtering works.
- ☒ Sorting works.
- ☒ Clear Search and Filters works.
- ☒ Result information updates.
- ☒ No-results state appears correctly.
- ☒ Appointment cards refresh automatically after adding another appointment.

CHECKPOINT

Before moving on, confirm that:

- ☒ Search works.
- ☒ Filter works.
- ☒ Sorting works.
- ☒ No-results state works.
- ☒ Result information works.
- ☒ Schedule refresh works.

COMMON BEGINNER MISTAKES

Many beginners request appointments from the database every time the user presses a key.

The application already has the appointment information.

Searching the existing appointment array is much faster and provides a smoother user experience.

BEHIND THE SCENES

The Appointment Schedule now behaves much like professional appointment management software.

The appointments are retrieved once.

JavaScript then performs searching, filtering and sorting locally inside the browser, making the interface feel much faster.

THINK LIKE A SOFTWARE DESIGNER

Good software does not make users adapt to the data.

Instead, it adapts the data to the way users naturally search for information.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building appointment search, filtering and sorting. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build live appointment search
- filter appointments
- sort appointments
- improve software performance
- create a professional appointment discovery experience

CHAPTER 7

SEARCHING, FILTERING AND SORTING APPOINTMENTS

LESSON 2

TESTING APPOINTMENT DISCOVERY

Estimated Time

45 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete appointment discovery system.

Rather than adding new functionality, you will confirm that searching, filtering and sorting continue working correctly under different situations.

WHY THIS MATTERS

Search features often work correctly when tested individually but fail when several controls are combined.

Professional testing ensures that every control behaves predictably.

BEFORE YOU CONTINUE

Prepare:

- Several Booked appointments
- Several Confirmed appointments
- Appointments with different names
- Appointments with different creation dates

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully before continuing.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Create several appointments with different clients, services, dates and statuses.

Confirm that:

- ☒ Client and service searches work.
- ☒ Email and phone searches work.
- ☒ Status filtering works.
- ☒ Today, Upcoming and Past filters use the correct local date.
- ☒ Every sorting option works.
- ☒ Search, filters and sorting work together.
- ☒ Result information is accurate.
- ☒ No-results is different from an empty schedule.
- ☒ Clear Search and Filters restores the complete schedule.
- ☒ New, edited and cancelled appointments refresh the results.
- ☒ Account privacy remains protected.

CHECKPOINT

Before completing this chapter, confirm that:

- ☒ Every Appointment Schedule control works.
- ☒ Combined controls work correctly.

- ✓ Appointments remain protected.
- ✓ Appointment cards never duplicate.

COMMON BEGINNER MISTAKES

- Some developers test every control separately but never test them together.
- Professional software should behave correctly regardless of how many controls the user combines.

BEHIND THE SCENES

- Every Appointment Schedule update follows the same sequence:
- Retrieve appointments once.
- Apply search.
- Apply filtering.
- Apply sorting.
- Display the final result.
- Using one predictable process makes the application easier to maintain.

THINK LIKE A SOFTWARE DESIGNER

- When several controls affect the same information, build one central workflow that coordinates them instead of allowing each control to work independently.

WHAT YOU LEARNED

- In this lesson you learned how to:
- test combined software features
- verify appointment discovery
- test responsive interfaces
- validate professional application behaviour

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System now allows users to locate appointment information quickly using professional search, filtering and sorting tools.

Instead of browsing long appointment lists manually, users can now find the information they need within seconds.

These improvements significantly increase the usability of the software while keeping appointment information secure.

CHAPTER MILESTONE

By the end of Chapter 7, you have successfully built:

- ✓ Appointment Schedule Toolbar
- ✓ Live Appointment Search
- ✓ Appointment Status Filtering
- ✓ Appointment Sorting
- ✓ Result Counter
- ✓ No-results State
- ✓ Clear Search and Filters
- ✓ Automatic Schedule Refresh
- ✓ Professional Appointment Discovery

TRANSITION TO CHAPTER 8

Your Appointment Booking System can now create, display and locate appointments efficiently.

The next step is allowing businesses to keep appointment information accurate as it changes.

Appointments move to new addresses.

Phone numbers change.

Companies grow.

Notes need updating.

In the next chapter, you will build a complete appointment editing workflow that allows authorised users to update appointment information securely while preserving data quality and appointment privacy.

CHAPTER 8

EDITING APPOINTMENTS

CHAPTER INTRODUCTION

Appointment plans sometimes change.

A client may request a different service, date or time.

Businesses also discover mistakes that need to be corrected after appointments have been created.

A professional Appointment Booking System must therefore allow authorised users to update appointment information safely.

Editing appointments introduces an important challenge.

The application must identify exactly which appointment is being edited, retrieve that appointment's information securely, prevent overlapping appointment times and ensure that one user can never modify another user's records.

In this chapter, you will build a complete appointment editing workflow that follows the same security principles used throughout this workbook.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Edit Appointment workflow
- Appointment edit mode
- Automatic form population
- Secure appointment retrieval for editing

- Appointment information updating
- Validation during editing
- Appointment field preparation
- Overlap protection during editing
- Secure database updates
- Cancel editing
- Automatic dashboard refresh
- Appointment editing security tests

LESSON 1

BUILDING APPOINTMENT EDITING

Estimated Time

120 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete appointment editing workflow.

Users will be able to open an appointment details, select Edit Appointment, automatically load the existing appointment information into the dashboard form, update the information and save the changes securely.

The application will reuse the existing appointment form instead of creating a separate editing form.

WHY THIS MATTERS

Professional software avoids unnecessary duplication.

Instead of maintaining one form for creating appointments and another for editing appointments, the same form can perform both tasks.

This reduces maintenance, keeps the user experience consistent and ensures that appointment information always passes through the same validation rules.

BEFORE YOU CONTINUE

Confirm that:

☒ Appointment details work correctly.

☒ Edit Appointment opens:

dashboard.html?edit=APPOINTMENT_ID

☒ Appointment creation still works.

☒ Appointment Schedule works.

☒ Dashboard statistics work.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

I already have a working Appointment Booking System.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- dashboard.html
- dashboard.js
- appointment-details.js
- auth.css

GENERAL GOAL

Build complete appointment editing using the existing booking form.

Edit Appointment should open:

dashboard.html?edit=APPOINTMENT_ID

Pass only the appointment ID.

When edit mode begins:

- get the authenticated user
- retrieve one appointment belonging to that user
- fill the existing form
- convert starts_at and ends_at into local date and time field values
- change the heading and submit button text
- display Cancel Edit

Reuse the existing validation.

When saving:

- rebuild complete ISO start and end values
- require the end to be later than the start

- update only the matching id and authenticated user_id
 - update updated_at
 - preserve id, user_id and created_at
- Continue relying on Row Level Security and the overlap constraint.
- If the new time overlaps another active appointment, show a friendly message.
- After success:
- show Appointment updated successfully.
 - leave edit mode
 - refresh the schedule
 - refresh statistics
 - preserve all search and schedule features
- Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- dashboard.html
- dashboard.js
- appointment-details.js
- auth.css

The application should now support complete appointment editing.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open:

Appointment Details

Select:

Edit Appointment

Confirm that:

- ☒ Edit mode begins.
- ☒ Appointment information loads.
- ☒ Form heading changes.
- ☒ Button text changes.

Modify:

- Phone
- Service
- Notes

Select:

Save Changes

Confirm that:

- ☒ Validation works.
- ☒ Overlap protection works.
- ☒ Appointment updates successfully.
- ☒ Dashboard statistics refresh.
- ☒ Appointment Schedule refreshes.

Open the profile again.

Confirm that the updated information appears.

CHECKPOINT

Before moving on, confirm that:

- ✓ Edit mode works.
- ✓ Appointment information loads.
- ✓ Validation still works.
- ✓ Overlap protection still works.
- ✓ Appointment updates successfully.
- ✓ Dashboard refreshes.

COMMON BEGINNER MISTAKES

Some beginners build a completely separate editing form.

Professional applications often reuse the same form for both creating and editing records.

This reduces duplicated code and keeps behaviour consistent.

BEHIND THE SCENES

The appointment form now supports two modes.

When no appointment ID exists, it creates a new appointment.

When an edit appointment ID exists, it updates the existing appointment.

The same interface now supports two different business workflows.

THINK LIKE A SOFTWARE DESIGNER

Whenever two features perform nearly the same task, look for opportunities to reuse existing components instead of creating new ones.

Reusable software is easier to maintain and usually contains fewer bugs.

CODE-READING QUESTION

Open dashboard.js. Find one function that supports building appointment editing. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build complete appointment editing
- reuse existing software components
- update appointments securely
- protect appointment ownership
- refresh business information automatically

CHAPTER 8

EDITING APPOINTMENTS

LESSON 2

TESTING APPOINTMENT EDITING

Estimated Time

50 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test the complete appointment editing workflow.

You will verify that appointment updates remain secure, overlap protection works correctly and appointment information cannot be modified across user accounts.

WHY THIS MATTERS

Editing changes existing business information.

Testing helps ensure that authorised users can update their own appointments without accidentally affecting another user's data.

BEFORE YOU CONTINUE

Prepare:

- Two user accounts.
- Several appointments.

Include overlapping and non-overlapping time scenarios.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Open an appointment and select Edit Appointment.

Confirm that:

- ☒ The correct appointment loads.
- ☒ Date and time fields show local values.
- ☒ Every existing value appears correctly.
- ☒ Validation still works.
- ☒ A valid edit saves.
- ☒ An overlapping edit is rejected.
- ☒ id, user_id and created_at do not change.
- ☒ updated_at changes.
- ☒ Cancel Edit leaves the appointment unchanged.
- ☒ Schedule and statistics refresh after success.
- ☒ Account A cannot edit Account B's appointment by changing the ID.

CHECKPOINT

Before completing this chapter, confirm that:

- ☒ Appointment editing works.
- ☒ Appointment ownership remains protected.

- ✓ Overlap protection works.
- ✓ Dashboard refreshes correctly.

COMMON BEGINNER MISTAKES

- Many beginners forget to exclude the appointment currently being edited during overlap protection.
- This causes the appointment to appear as a duplicate of themselves.
- Professional editing workflows always exclude the current record before checking for duplicates.

BEHIND THE SCENES

- Appointment editing follows the same sequence used during appointment creation.
- The difference is that the database performs an UPDATE instead of an INSERT.
- Every other quality and security check remains the same.

THINK LIKE A SOFTWARE DESIGNER

- Consistency makes software easier to understand.
- Whenever possible, editing should follow the same rules as creating new records.

WHAT YOU LEARNED

- In this lesson you learned how to:
- test appointment editing
- verify secure updates
- verify overlap protection
- protect appointment ownership

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System now allows authorised users to update appointment information securely.

The application reuses the existing appointment form, validates updated information, prevents overlapping appointment times and refreshes the dashboard automatically after successful updates.

The editing workflow now behaves like that found in many professional appointment management systems.

CHAPTER MILESTONE

By the end of Chapter 8, you have successfully built:

- ✓ Appointment Edit Mode
- ✓ Secure Appointment Retrieval
- ✓ Automatic Form Population
- ✓ Appointment Updating
- ✓ Validation During Editing
- ✓ Overlap Protection During Editing
- ✓ Secure Database Updates
- ✓ Cancel Edit
- ✓ Automatic Dashboard Refresh
- ✓ Appointment Editing Security

TRANSITION TO CHAPTER 9

Your Appointment Booking System can now create, display, locate and update appointment information.

The final stage of appointment management is removing information that is no longer needed.

In the next chapter, you will build secure appointment cancellation while protecting users from accidentally removing important business information.

CHAPTER 9

CANCELLING APPOINTMENTS

CHAPTER INTRODUCTION

Appointment plans sometimes change.

A client may ask to cancel, or the business may no longer be available at the selected time.

This application will use soft cancellation.

The appointment will remain in the database, but its status will change to cancelled.

This preserves useful history, keeps statistics honest and releases the time for a new booking.

The user will always confirm the action, and the database will verify ownership before making the change.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will build:

- Cancel Appointment action
- Clear confirmation dialog
- Secure cancel_appointment call
- Soft cancellation
- Preserved appointment history
- Released booking time
- Schedule and statistics refresh
- Two-account cancellation testing

LESSON 1

BUILDING SECURE APPOINTMENT CANCELLATION

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will build the complete soft-cancellation workflow.

The user will confirm the decision, the secure database function will change the appointment status, and the appointment will remain in the schedule as useful business history.

WHY THIS MATTERS

Cancellation should be deliberate and reversible only through a later extension.

The application must ask for confirmation, preserve the record and verify authenticated ownership before changing the status.

BEFORE YOU CONTINUE

Confirm that:

- ☒ Appointment editing works.
- ☒ Appointment details work.
- ☒ Dashboard statistics work.
- ☒ Appointment Schedule works.
- ☒ secure cancellation function exists on the appointments table.
- ☒ Row Level Security remains enabled.

BUILD PROMPT

PROMPT STARTS HERE

PROJECT STATE

The Appointment Details page already loads one authorised appointment.

The database already contains the `cancel_appointment` function.

Everything already built must continue working.

Return complete updated files only.

Do not return snippets or partial code.

Update only:

- `appointment-details.html`
- `appointment-details.js`
- `dashboard.js`
- `auth.css`

GENERAL GOAL

Build secure soft cancellation.

When Cancel Appointment is selected:

Display a confirmation dialog:

Are you sure you want to cancel this appointment?

The appointment will remain in the schedule as Cancelled.

If the user chooses not to continue:

- stop immediately
- do not contact Supabase

If confirmed:

- call `cancel_appointment` with the current appointment ID
- do not use a DELETE request
- do not remove the record permanently
- continue relying on authentication, Row Level Security and the secure function

After success:

- display Appointment cancelled successfully.
 - update the details status
 - disable the Cancel Appointment action
 - refresh the schedule
 - refresh dashboard statistics
- Show a friendly message for errors and send technical details to `console.error()`.
- Style the cancelled state clearly without making it look like an unexpected error.
- Return complete updated files only.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

ChatGPT should return complete updated versions of:

- appointment-details.html
- appointment-details.js
- dashboard.js
- auth.css

The application should now support secure appointment cancellation.

SAVE YOUR FILES

BACK UP BEFORE REPLACING FILES

Before replacing an existing file, copy your complete project folder.

Save the copy outside your working project folder.

Name the backup clearly using the current chapter and lesson.

Open the backup and confirm that your files are inside it.

CONTINUE SAVING YOUR FILES

Replace the complete contents of every updated file.

Save each file.

TEST YOUR WORK

Open an active appointment and select Cancel Appointment.

First, stop at the confirmation dialog.

Confirm that the appointment remains unchanged.

Open the confirmation again and continue.

Verify that:

- ☒ Status changes to cancelled.
- ☒ The appointment remains in the schedule.
- ☒ The details page updates.
- ☒ Statistics refresh.
- ☒ The same appointment cannot be cancelled twice.
- ☒ Another user's appointment cannot be cancelled.

CHECKPOINT

Before moving on, confirm that:

- ☒ Confirmation works.
- ☒ Appointment cancellation works.
- ☒ Dashboard refreshes.
- ☒ Statistics refresh.
- ☒ Appointment ownership remains protected.

COMMON BEGINNER MISTAKES

A common mistake is removing the appointment row permanently.

This workbook keeps the record and changes only its status.

Another mistake is sending only an appointment ID without checking the authenticated owner.

The secure function and Row Level Security provide that ownership protection.

BEHIND THE SCENES

The confirmation protects against an accidental click.

The secure function protects ownership.

The status change preserves history and releases the time because the overlap constraint ignores cancelled appointments.

THINK LIKE A SOFTWARE DESIGNER

A cancelled appointment is still useful information.

It can help a business understand cancellation patterns and keep a truthful record of what happened.

Design important business actions around history, clarity and security.

CODE-READING QUESTION

Open `appointment-details.js`. Find one function that supports building secure appointment cancellation. What is the function called, and what action makes it run?

Write your answer in your own words.

WHAT YOU LEARNED

In this lesson you learned how to:

- build secure appointment cancellation
- protect appointment ownership
- confirm destructive actions
- refresh application data automatically

CHAPTER 9

CANCELLING APPOINTMENTS

LESSON 2

TESTING APPOINTMENT CANCELLATION

Estimated Time

40 minutes

WHAT YOU ARE BUILDING

In this lesson, you will test soft cancellation from beginning to end.

You will confirm that the status changes safely, the record remains visible, the released time can be booked again and account privacy remains protected.

WHY THIS MATTERS

Cancellation affects the details page, schedule, overlap protection and statistics.

Testing every connected result confirms that the complete capability works, not only the button.

BEFORE YOU CONTINUE

Prepare:

- Two authenticated user accounts.
- Several appointments.

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated.

TEST YOUR WORK

Test cancellation carefully.

Confirm that:

- ☒ Choosing Cancel in the confirmation dialog changes nothing.
- ☒ Confirming changes the status to cancelled.
- ☒ The appointment remains in the schedule.
- ☒ The details page shows Cancelled.
- ☒ The cancellation button cannot cancel the same appointment again.
- ☒ Today's, upcoming and cancelled statistics refresh.
- ☒ The cancelled time becomes available for another booking.
- ☒ No DELETE request is used.
- ☒ Account A cannot cancel Account B's appointment.
- ☒ Row Level Security remains enabled.

CHECKPOINT

Before completing this chapter, confirm that:

- ☒ Appointment cancellation works.
- ☒ Confirmation works.
- ☒ Dashboard refreshes.

✓ Appointment ownership remains protected.

✓ Dashboard statistics remain accurate.

COMMON BEGINNER MISTAKES

A common mistake is testing only the successful confirmation.

Also test stopping at the confirmation dialog, trying to cancel twice and attempting cross-account access.

Each test protects a different part of the workflow.

BEHIND THE SCENES

Soft cancellation changes one status, but several parts of the application respond.

The schedule changes its visual label, upcoming statistics change, cancelled statistics increase and the time becomes available again.

THINK LIKE A SOFTWARE DESIGNER

A business capability is complete only when its connected results are correct.

Always test what changed, what remained and what became possible after the action.

WHAT YOU LEARNED

In this lesson you learned how to:

- test secure cancellation
- verify ownership protection
- verify dashboard updates
- verify application consistency

CHAPTER SUMMARY

Congratulations.

Your Appointment Booking System now supports the complete appointment lifecycle.

Users can create, view, search, update and cancel appointments securely.

Every operation respects authentication, appointment ownership and Row Level Security.

The application now behaves like a complete appointment management solution suitable for many small businesses.

CHAPTER MILESTONE

By the end of Chapter 9, you have successfully built:

- ✓ Secure Appointment Cancellation
- ✓ Cancel Confirmation
- ✓ Protected Appointment Ownership
- ✓ Automatic Dashboard Refresh
- ✓ Automatic Appointment Schedule Refresh
- ✓ Professional Cancellation Workflow
- ✓ Appointment Cancellation Testing

TRANSITION TO CHAPTER 10

Your Appointment Booking System is now functionally complete.

The final chapter focuses on ensuring that every feature works together reliably.

You will perform a complete end-to-end review of the application, verify every business workflow and prepare the software for deployment.

CHAPTER 10

FINAL APPLICATION REVIEW

CHAPTER INTRODUCTION

Congratulations.

You have successfully built a complete Appointment Booking System from the ground up.

Your application now supports the complete appointment lifecycle.

Users can:

- Register
- Verify their email address
- Sign in securely
- Create appointments
- View appointments
- Search and organise appointments
- Edit appointment information
- Cancel appointments

Every feature has been built using HTML, CSS, JavaScript and Supabase without relying on a traditional software development framework.

Before publishing software for real users, professional developers perform a complete system review.

Rather than testing individual features in isolation, they test the entire application from beginning to end.

In this chapter, you will do exactly the same.

WHAT YOU WILL BUILD IN THIS CHAPTER

During this chapter, you will:

- Perform complete end-to-end testing
- Verify appointment security
- Verify Row Level Security
- Test responsive layouts
- Test application usability
- Prepare the application for deployment

LESSON 1

COMPLETE SYSTEM TESTING

Estimated Time

90 minutes

WHAT YOU ARE BUILDING

In this lesson, you will perform a complete review of your Appointment Booking System.

Instead of testing one feature at a time, you will test the application exactly as a real business user would.

By the end of this lesson, you should have complete confidence that every feature works correctly before publishing your software.

WHY THIS MATTERS

Software is only valuable when all its individual parts work together reliably.

Professional software development does not end when the last feature is written.

It ends when the complete application has been tested thoroughly.

BEFORE YOU CONTINUE

Prepare:

- Two user accounts
- Several Booked appointments
- Several Confirmed appointments
- Appointments with and without client email addresses
- Appointments with complete and incomplete optional information

BUILD PROMPT

PROMPT STARTS HERE

There is nothing to build with ChatGPT in this lesson.

Complete every test carefully.

If you discover a problem, correct it before deploying your application.

PROMPT ENDS HERE

WHAT AI SHOULD RETURN

Nothing.

SAVE YOUR FILES

No files are created or updated during this lesson.

TEST YOUR WORK

Complete the following final tests on the deployed HTTPS website.

AUTHENTICATION

- ✓ Registration, email verification, login and logout work.
- ✓ Protected pages reject signed-out visitors.

BOOKING

- ✓ Required fields are enforced.
- ✓ Local date and time values save correctly.
- ✓ An end time before the start is rejected.
- ✓ Overlapping active appointments are rejected.

SCHEDULE

- ✓ Appointments appear in time order and under the correct date.
- ✓ Local times, statuses and empty states display correctly.

SEARCH AND FILTERS

- ✓ Search, date filters, status filters and sorting work together.

EDITING

- ✓ A valid edit saves securely.
- ✓ An overlapping edit is rejected.

CANCELLATION

- ✓ Cancelling keeps the record and changes its status.
- ✓ A cancelled time can be booked again.

STATISTICS

- ✓ Total, today, upcoming and cancelled counts remain accurate.

SECURITY

- ✓ Account A cannot view, edit or cancel Account B's appointment.
- ✓ Row Level Security remains enabled.

RESPONSIVE DESIGN

- ✓ The form, schedule, toolbar and details page work on a small screen.

CONSOLE

- ✓ No unexpected red errors appear during the complete workflow.

CHECKPOINT

Before publishing your software, confirm that:

- ✓ Every feature works.
- ✓ Authentication works.
- ✓ Appointment ownership is protected.
- ✓ Dashboard statistics remain accurate.
- ✓ Appointment Schedule remains accurate.
- ✓ Appointment details remain accurate.
- ✓ Responsive layouts work correctly.
- ✓ No unexpected JavaScript errors appear.

COMMON BEGINNER MISTAKES

Some developers stop testing after the application appears to work once.

Professional developers continue testing until they are confident that users can complete every important workflow successfully.

Another common mistake is testing only with one user account.

Always test with multiple accounts to confirm that appointment privacy remains protected.

BEHIND THE SCENES

Every feature in your Appointment Booking System depends on the others.

Authentication protects appointment information.

Appointment creation supplies the schedule.

The schedule opens appointment details.

Appointment editing updates those appointments.

Appointment cancellation removes them.

Dashboard statistics reflect every change.

Testing confirms that these connected features continue working together.

THINK LIKE A SOFTWARE DESIGNER

Software development is not simply about writing code.

It is about delivering reliable software that people can trust.

Every hour spent testing before deployment saves many hours of fixing problems later.

WHAT YOU LEARNED

In this lesson you learned how to:

- perform complete application testing
- verify appointment security
- verify software reliability
- prepare software for deployment

FINAL TESTING

Before considering this workbook complete, perform one final review.

Create a completely new user account on the deployed HTTPS website.

Begin with an empty appointment schedule.

Complete this journey:

- ✓ Register
- ✓ Verify the email address
- ✓ Sign in
- ✓ Book appointments on different dates
- ✓ Try and fail to create an overlapping appointment
- ✓ View the complete schedule
- ✓ Search, filter and sort appointments
- ✓ Open one appointment
- ✓ Edit one appointment

- ✓ Cancel one appointment
- ✓ Confirm the dashboard statistics
- ✓ Sign out
- ✓ Sign in again

Confirm that every saved appointment still exists and every statistic remains correct.

If every step succeeds, your Appointment Booking System is ready to be published.

GOING LIVE

Your Appointment Booking System is ready for its final deployment.

1.

Save every complete project file.

2.

Open:

<https://app.netlify.com/drop>

3.

Drag the complete Appointment Booking System folder into the deployment area.

If you already created the Netlify website during authentication testing, deploy the updated folder to that same website.

4.

Wait for deployment to finish.

5.

Open the HTTPS website address supplied by Netlify.

6.

Confirm that the Supabase Site URL and Redirect URLs still use this exact website address.

7.

Repeat the complete live workflow:

- Registration and email verification
- Appointment booking

- Schedule viewing
- Search, filters and sorting
- Editing
- Cancellation
- Dashboard statistics
- Two-account privacy

Only share the application after every important live test succeeds.

PORTFOLIO DESCRIPTION

Congratulations.

You have successfully built a fully functional Appointment Booking System.

Your application demonstrates practical software development skills including:

- User authentication
- Database design
- Secure appointment management
- CRUD operations
- Dashboard design
- Search and filtering
- Responsive layouts
- Database security using Row Level Security

This project is an excellent addition to your software development portfolio because it solves a genuine business problem and demonstrates that you can build secure, data-driven applications using Artificial Intelligence.

REFLECTION QUESTIONS

Before moving to the next workbook, take a few minutes to think about what you have learned.

1.

Which part of this project helped you understand software development better?

2.

How did AI help you build this application more efficiently?

3.

Which software development concepts now make more sense than they did before beginning this workbook?

4.

If you were building this application for a real business, what additional features would you consider adding?

5.

What part of the project challenged you the most, and how did you solve it?

Writing down your answers will help reinforce what you have learned.

EXTENSION CHALLENGES

If you would like to improve your Appointment Booking System further, consider adding some of these features.

- Appointment details photographs
- Appointment tags
- Favourite appointments
- Appointment birthdays
- Appointment interaction history
- Appointment follow-up reminders
- Appointment document uploads
- Appointment export to CSV
- Appointment import from CSV
- Appointment activity log

Do not attempt to build every feature immediately.

Choose one feature at a time and treat it as a separate software project.

NEXT WORKBOOK

Excellent work.

You have completed Workbook 05 of the Prompt to Profit™ Software Workbook Series.

You have built software that allows a business to book, view, find, edit and cancel appointments.

You have also protected every appointment using authentication, Row Level Security and database rules that prevent overlapping active bookings.

The next project in the series is Workbook 06 — Sales Tracker.

For now, review what you have built and make sure every important feature works correctly.

Congratulations once again on completing your Appointment Booking System.